



JOKU OTSIKKO

Omar Mayani

Pro gradu -tutkielma
Kesäkuu 2020

MATEMATIIKAN JA TILASTOTIETEEN LAITOS

Tarkastajat:

Prof. Ion Petre

Markus Ylikojola

Turun yliopiston laatu järjestelmän mukaisesti tämän julkaisun alkuperäisyys on tarkastettu Turnitin OriginalityCheck-järjestelmällä

TURUN YLIOPISTO, Matematiikan ja tilastotieteen laitos

Pro gradu -tutkielma

Pääaine: Matematiikka

Tekijä: Omar Mayani

Otsikko: Joku otsikko

Ohjaaja: Prof. Ion Petre

Sivumäärä: xx sivua + liitteet xx sivua

Aika: Kesäkuu 2020

Kirjoita tähän tiivistelmä. Laita `\noindent`-komento kappaleen alkuun, niin $\text{\LaTeX}$ ei sisennä ensimmäistä riviä.

Komento `\vspace` taas jättää sopivan välin kappaleiden väliin - 4mm näyttää aika hyvältä.

Kirjoita tiivistelmä napakasti ja kaikenlaista toistoa välttäen.

Asiasanat: tiivistelmäsiivu, Pro gradu -tutkielma, $\text{\LaTeX}$ -ladontajärjestelmä.



Teknologiateollisuus

Tämän pro gradu -tutkielman tutkimustyötä on tuettu Teknologiateollisuuden 100-vuotissäätiön myöntämällä apurahalla.

ON NOTATION AND TERMINOLOGY

Unless otherwise specified, we use the following notations and terminologies throughout the thesis.

$\mathbb{R}$	the set of real numbers,
J	a loss function,
$\mathbf{T}$	the transpose of a matrix,
w	a member of a vocabulary (usually a word),
$\mathcal{V} = \{w_1, w_2, \dots, w_{ \mathcal{V} }\}$	a (finite) vocabulary,
$\mathbf{w}$	the vector representation of w ,

Sisällys

1	Johdanto	4
2	Diskreetit tekstiesitykset ja klassinen kielimallinnus	7
2.1	Tokenien one-hot-enkoodaus	7
2.2	N-gram-mallit	8
3	Jatkuvat esitykset ja neuroverkkopohjainen kielimallinnus	10
3.1	Upotusvektorit	10
3.2	Upotusvektorien oppiminen Word2Vec-menetelmällä	11
3.3	N-gram-malli optimointiongelmana	13
3.4	Monikerroksinen perseptroni seuraavan tokenin ennustajana	15
4	Gradienttimenetelmä	18
4.1	Häiriöalttius ja toisen kertaluvun geometria	21
4.2	Adaptiiviset optimointimenetelmät	22
5	Dekooderipohjainen Transformer-arkkitehtuuri suurille kielimalleille	26
5.1	Itsehuomio	26
5.2	Positiokoodaus	29
5.3	Dekooderikerros	31
5.4	Dekooderikerroksista seuraavan tokenin jakaumaan	32
5.5	Autoregressiivinen koulutus ja generointi	33
5.6	Kielimalli Text-to-SQL-tehtävässä	35
6	Metodologia	38
6.1	Tutkimusasetelma	38
6.2	Tutkimusaineisto	38
6.3	Cortex Analyst ja semanttinen näkymä	38
6.4	Arviointiperusteet	40
7	Kokeet	42
8	Tulokset	45
9	Rajoitukset	50
10	Johtopäätökset	51

1 Johdanto

Kielimalli on autoregressiivinen malli, joka ennustaa seuraavan sanan (tai tokenin) aiemman kontekstin perusteella [17, 3]. Kun mallin tuottama sana liitetään aiemman kontekstiin, kutsutaan mallia päivitetyllä kontekstilla, jolloin se voi tuottaa pitkiäkin tekstijaksoja. Suuret kielimallit ovat viime vuosina kehittyneet pelkistä tekstin tuottajista järjestelmiksi, jotka kykenevät myös hyödyntämään ulkoisia työkaluja [34, 48]. Jos kielimallia suorittava ympäristö tulkitsee tiettyjä mallin tuottamia merkintöjä työkalukutsuina, kielimalli voi käyttää esimerkiksi laskentaa, verkkohakua tai tietokantakyselyitä.

Luonnollisen kielen käyttö tietokantojen rajapintana (Text-to-SQL) on kiinnostava sovelluskohde, koska se voi madaltaa datan hyödyntämisen kynnystä [21, 10]. Tällöin käyttäjän ei tarvitse osata SQL-kieltä eikä tuntea tietokannan rakennetta yksityiskohtaisesti, vaan hän voi esittää tiedontarpeensa tavallisella kielellä. Käytännön hyöty riippuu kuitenkin siitä, kuinka luotettavasti järjestelmä kykenee tulkitsemaan käyttäjän tarkoituksen ja muuntamaan sen oikeaksi SQL-kyselyksi [49, 19].

Tässä tutkielmassa tarkastellaan Snowflaken Cortex Analyst -järjestelmää taustatutkimuksena [39]. Cortex Analyst on kielimallipohjainen työkalu, joka muodostaa käyttäjän luonnollisen kielen kysymyksistä SQL-kyselyitä ja voi suorittaa niitä tietokantaa vasten. Tutkimuskohteena on puhelinkeskusdataa sisältävä tietokanta, jossa analyysin keskiössä on yksi keskusteludataa sisältävä päätason taulu sekä kolme sitä täydentävää taulua (ks. kuva 1). Järjestelmän toiminta perustuu semanttiseen näkymään, jossa taulut, kentät ja niiden merkitykset kuvataan YAML-muodossa (ks. kuva 2) [38]. Tämä näkymä toimii linkkinä käyttäjän luonnollisen kielen kysymyksen ja tietokannan rakenteen välillä.

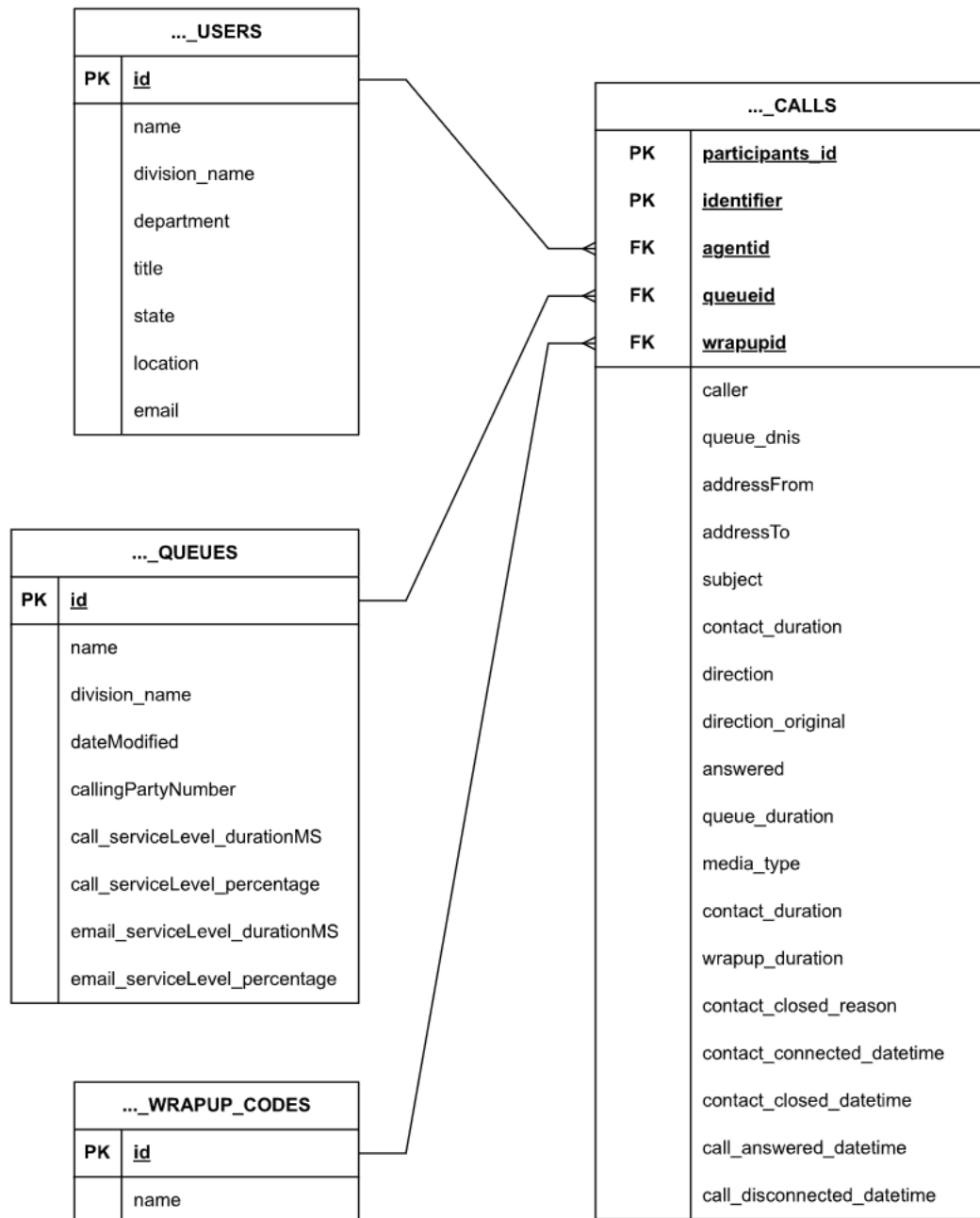
Tutkielman tavoitteena on arvioida, kuinka hyvin Cortex Analyst soveltuu luonnollisen kielen tietokantakyselyihin ja millaisia rajoituksia sen käyttöön liittyy. Koska nykyaikaisten kaupallisten kielimallien tarkat toteutustiedot eivät yleensä ole julkisia, järjestelmän arviointi perustuu empiiriseen tarkasteluun [8]. Tässä työssä vertaillaan nimenomaan Cortex Analystin tuottamia SQL-kyselyitä eikä niiden palauttamia vastauksia [27]. Raja on perusteltu sekä aineiston luottamuksellisuuden vuoksi että siksi, että virheellisen kyselyn vaikutus lopulliseen vastaukseen riippuu myös tietokannan sisällöstä.

Tutkimusta ohjaavat seuraavat tutkimuskysymykset: Kuinka konsistentisti Cortex Analyst tuottaa SQL-kyselyitä, kun sama kysymys esitetään useita kertoja? Kuinka herkkä Cortex Analyst on sellaisille syötteen muutoksille, joiden ei pitäisi muuttaa kysymyksen merkitystä? Millaisia virhe- ja vaihtelutyyppejä järjestelmän tuottamissa SQL-kyselyissä esiintyy?

Tutkielma etenee siten, että ensin kuvataan tutkimusasetelma, aineisto ja arviointitapa. Tämän jälkeen esitellään kokeet, joissa tarkastellaan järjestelmän konsistenssia ja herkkyyttä merkitykseltään vähäisille muutoksille. Lopuksi esitetään tulokset, johtopäätökset ja tutkimuksen keskeiset rajoitukset.

Attribute	Value
PARTICIPANTS_ID	ea0a918f
IDENTIFIER	63c44469
QUEUEID	3b10a230
CALLERS	+123456789012
AGENTID	b7e632de
SUBJECT	Null
ADDRESS_TO	Null
ADDRESS_FROM	Null
CONTACT_DURATION	363
DIRECTION	inbound
DIRECTION_ORIGINAL	inbound
ANSWERED	1
QUEUE_DURATION	2
CALL_DURATION	316
WRAPUPID	dd91b10d
WRAPUP_DURATION	47
CONTACT_CLOSED_REASON	client
CONTACT_CONNECTED_DATETIME	2025-08-23 13:41:13.000
CONTACT_CLOSED_DATETIME	2025-08-23 13:47:28.000
CALL_ANSWERED_DATETIME	2025-08-23 13:41:25.000
CALL_DISCONNECTED_DATETIME	2025-08-23 13:46:41.000
MEDIA_TYPE	call
CONTACT_CONNECTED_DATE	2025-08-23 00:00:00.000

Taulukko 1: Esimerkki yhdestä tietokannan rivistä. Osa kentistä on anonymisoitu tietosuoja- ja luottamuksellisuussyistä.



Kuva 1: Tutkimuksessa käytettävän tietokannan rakenne. Taulujen nimet on lyhennetty luettavuuden vuoksi.

2 Diskreetit tekstiesitykset ja klassinen kielimallinnus

Kielimallien yhteydessä tekstiä käsitellään tavallisesti diskreetteinä tokeneina, jotka muunnetaan laskennalliseen muotoon mallintamista varten. Olkoon $\mathcal{V} = \{v_1, v_2, \dots, v_{|\mathcal{V}|}\}$ sanasto eli joukko kaikista mallin käyttämistä tokeneista ja olkoon $\mathcal{W} = (w_1, w_2, \dots, w_{|\mathcal{W}|})$ korpus eli havaittu tokenijono. Tällöin $w_t \in \mathcal{V}$ merkitsee korpuksen ajanhetken t tokenia. Tokenilla voidaan tarkoittaa esimerkiksi sanaa, osasanaa, merkkiä tai muuta diskreettiä tekstiyksikköä riippuen käytetystä tokenisointimenetelmästä [17].

2.1 Tokenien one-hot-enkoodaus

Diskreetit tokenit esitetään numeerisessa muodossa, jotta niitä voidaan hyödyntää tilastollisissa malleissa ja neuroverkoissa. Yksinkertaisin ja yleisesti käytetty esitystapa on one-hot-enkoodaus. Siinä kukin token $v_j \in \mathcal{V}$ esitetään vektorina $x(v_j) \in \{0, 1\}^{|\mathcal{V}|}$, jossa

$$x(v_j)_k = \begin{cases} 1, & \text{jos } k = j, \\ 0, & \text{muulloin.} \end{cases}$$

Tällainen esitys ilmaisee ainoastaan tokenin identiteetin: eri tokeneilla on toisistaan poikkeavat vektoriesitykset, mutta niihin ei sisälly eksplisiittistä semanttista tai syntaktista rakennetta [23]. Menetelmän etuna on yksikäsitteisyys ja sen yksinkertainen yhteys sanaston indeksointiin.

Käytännön kielimallinnuksessa sanastoa täydennetään usein erikoistokeneilla, joilla kuvataan sekvenssin rakennetta ja käsitellään harvinaisia havaintoja. Tällaisia ovat esimerkiksi aloitus- ja lopetusmerkit $\langle s \rangle$ ja $\langle /s \rangle$ sekä tuntemattoman tokenin merkki $\langle unk \rangle$ [17].

Aloitus- ja lopetusmerkit ovat erityisen hyödyllisiä silloin, kun mallin ennustettavaa kontekstia halutaan määritellä myös sekvenssin reuna-alueilla. Esimerkiksi n -gram-malleissa (ks. osio 2.2) kontekstin pituus on vakio, minkä vuoksi sekvenssin alku- ja loppupäähän lisätään tyypillisesti riittävä määrä erikoistokeneita, jotta kontekstit voidaan muodostaa yhtenäisesti kaikille ajanhetkille.

Koko korpus voidaan esittää one-hot-vektoreiden jonona

$$X = (x_1, x_2, \dots, x_{|\mathcal{W}|}),$$

missä $x_t = x(w_t)$ on tokenin w_t one-hot-esitys. Jos syöte koostuu useasta tokenista, voidaan niiden one-hot-esitykset yhdistää esimerkiksi ketjuttamalla ne yhdeksi syötevektoriksi

$$\tilde{x}_t = [x_{t-n+1}; x_{t-n+2}; \dots; x_{t-1}] \in \{0, 1\}^{(n-1)|\mathcal{V}|}.$$

Tämä muodostaa luonnollisen syötteen monille kontekstipohjaisille malleille, sillä se säilyttää kontekstin tokenijärjestyksen eksplisiittisesti.

One-hot-enkoodaus muodostaa perustavanlaatuisen sillan tekstin ja numeeristen kielimallien välille. Se tarjoaa täsmällisen ja muodollisesti yksinkertaisen esitystavan, jonka varaan voidaan rakentaa sekä tilastollisia n -gram-malleja että myöhempiä neuroverkkopohjaisia kielimalleja.

2.2 N-gram-mallit

N-gram-malli on klassinen tilastollinen kielimalli, jossa sekvenssin seuraavan tokenin todennäköisyyttä estimoidaan rajallisen kontekstin perusteella [17]. Menetelmä perustuu $(n - 1)$:n kertaluvun Markov-oletukseen, jonka mukaan tokenin w_t todennäköisyys riippuu vain sitä välittömästi edeltävistä $(n - 1)$ tokenista eikä koko aiemmasta historiasta. Markov-oletuksen perusteella ehdollinen todennäköisyys voidaan approksimoida muodossa

$$P(w_t \mid w_1, w_2, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1}),$$

missä w_t on ajanhetken t tokeni ja $(w_{t-n+1}, \dots, w_{t-1})$ muodostaa sitä edeltävän $(n - 1)$ tokenin kontekstin.

N-gram-mallien historialliset juuret yhdistetään usein Shannonin varhaiseen informaatioteoreettiseen työhön, jossa luonnollista kieltä tarkasteltiin symbolijonojen tilastollisena prosessina ja eri kertalukujen approksimaatioita havainnollistettiin englannin kielellä [35, 36].

Olkoon $\mathcal{V} = \{v_1, v_2, \dots, v_{|\mathcal{V}|}\}$ sanasto, jossa $|\mathcal{V}|$ on sanaston koko, ja $\mathcal{W} = (w_1, w_2, \dots, w_{|\mathcal{W}|})$ havaintoaineisto. N-gram-malli voidaan toteuttaa muodostamalla lukumatriisi $C \in \mathbb{R}^{|\mathcal{V}|^{n-1} \times |\mathcal{V}|}$, jossa $C_{c,j}$ edustaa sitä, kuinka monta kertaa kontekstia c seuraa tokeni v_j havaintoaineistossa. Formaalisti

$$C_{c,j} = |\{t \in \mathbb{N} \mid (w_{t-n+1}, \dots, w_{t-1}) = c \text{ ja } w_t = v_j\}|.$$

Kun lukumatriisin C rivit normalisoidaan jakamalla jokainen alkio rivinsä summalla, saadaan frekvenssimatriisi $F \in \mathbb{R}^{|\mathcal{V}|^{n-1} \times |\mathcal{V}|}$, jossa

$$F_{c,j} = \frac{C_{c,j}}{\sum_{k=1}^{|\mathcal{V}|} C_{c,k}},$$

ja joka edustaa ehdollista todennäköisyyttä

$$P(w_t = v_j \mid (w_{t-n+1}, \dots, w_{t-1}) = c).$$

Tekstin generointi n-gram-mallilla tapahtuu iteratiivisesti hyödyntämällä frekvenssimatriisia F . Ensin valitaan alkukonteksti, esimerkiksi erikoismerkeillä aloitettu $(n - 1)$ -tokenin pituinen alkujakso. Tämän jälkeen mallin tämänhetkistä kontekstia vastaava rivi F_c , tulkitaan todennäköisyysjakaumaksi, josta seuraava tokeni valitaan satunnaisotannalla. Kun tokeni w_t on generoitu, konteksti päivitetään siirtämällä ikkuna yhden askeleen eteenpäin siten, että uusi konteksti muodostuu edeltävistä $(n - 1)$ generoidusta tokenista. Prosessia jatketaan, kunnes generoidaan pääteimerkki tai saavutetaan haluttu tekstin pituus.

Vaikka lähestymistapa on käsitteellisesti yksinkertainen ja tulkittava, siihen liittyy merkittäviä laskennallisia haasteita erityisesti kontekstikoon n kasvaessa. Tämän seurauksena lukumatriisista $C \in \mathbb{R}^{|\mathcal{V}|^{n-1} \times |\mathcal{V}|}$ tulee nopeasti erittäin suuri ja käytännöllisissä sovelluksissa hyvin harva, koska vain pieni osa kaikista mahdollisista konteksteista esiintyy tyypillisessä havaintoaineistossa. Tämä lisää sekä muistin- että laskentatehon tarvetta [17].

Diskreetit esitykset ja frekvenssipohjaiset n -gram-mallit tarjoavat käsitteellisesti selkeän lähtökohdan kielimallinnukselle. Niiden keskeinen rajoite on kuitenkin heikko yleistyskyky harvinaisiin ja havaitsemattomiin konteksteihin astuen esiin niin sanottuna "ulottuvuuksien kirouksena" (engl. *curse of dimensionality*) [3]. Tämän vuoksi on hyödyllistä tarkastella samaa ongelmaa optimointitehtävänä, mikä muodostaa luontevan siirtymän kohti jatkuvia vektoriesityksiä ja parametrisoituja neuroverkko-pohjaisia malleja.

3 Jatkuvat esitykset ja neuroverkkopohjainen kielimallinnus

3.1 Upotusvektorit

One-hot-esitys tarjoaa yksikäsitteisen tavan kuvata tokeneita, mutta sen heikkoutena on, ettei se sisällä eksplisiittistä tietoa tokenien välisistä suhteista. Jokaisen tokenin vektorikuvaus on ortogonaalinen kaikkiin muihin nähden, jolloin esimerkiksi sanan *kissa* suhde sanaan *koira* on rakenteellisesti sama kuin sen suhde sanaan *tietokone*. Tämän rajoitteen ylittämiseksi käytetään upotusvektoreita (engl. *embedding vectors*), joissa kukin token kuvataan tiheällä reaaliarvoisella vektorilla.

Olkoon edelleen $\mathcal{V} = \{v_1, v_2, \dots, v_{|\mathcal{V}|}\}$ sanasto. Tällöin tokenille v_j määritellään upotus

$$e(v_j) \in \mathbb{R}^{1 \times d},$$

missä $d \ll |\mathcal{V}|$ on upotustilan dimensio.

Upotusvektorit voidaan esittää myös matriisimuodossa. Olkoon

$$E \in \mathbb{R}^{|\mathcal{V}| \times d}$$

upotusmatriisi, jonka rivit vastaavat sanaston tokeneita. Kun $x(v_j) \in \mathbb{R}^{1 \times |\mathcal{V}|}$ on tokenin v_j one-hot-esitys, sen upotus saadaan muodossa

$$e(v_j) = x(v_j)E \in \mathbb{R}^{1 \times d}.$$

Koska one-hot-vektori sisältää täsmälleen yhden nollasta poikkeavan komponentin, tämä kertolasku palauttaa suoraan tokenia v_j vastaavan upotusmatriisin rivin.

Upotusvektorien keskeinen etu liittyy siihen, että samankaltaisten tokenien upotukset voivat sijaita lähellä toisiaan upotustilassa. Tällöin tokenien geometrinen läheisyys voi heijastaa niiden semanttista tai syntaktista samankaltaisuutta. Esimerkiksi sanat “kuningas” ja “kuningatar” voivat saada upotukset, jotka ovat lähellä toisiaan, koska ne esiintyvät samankaltaisissa konteksteissa ja jakavat merkityksellisiä piirteitä. Upotusvektorit tarjoavat siten rikkaamman ja joustavamman esityksen tokenien välisistä suhteista kuin one-hot-esitys, jossa kaikki tokenit ovat yhtä kaukana toisistaan.

Käytännön kielimallinnuksessa upotukset opitaan tavallisesti tehtäväkohtaisesti osana laajempaa optimointia. Jos korpus on $\mathcal{W} = (w_1, w_2, \dots, w_{|\mathcal{W}|})$, voidaan jokainen tokeni $w_t \in \mathcal{V}$ korvata sitä vastaavalla upotusvektorilla $e(w_t)$. Tällöin koko korpusta voidaan tarkastella tiheiden vektorien jonona

$$E_{\mathcal{W}} = (e(w_1), e(w_2), \dots, e(w_{|\mathcal{W}|})).$$

Tämä esitystapa on erityisen hyödyllinen neuroverkoissa, joissa upotukset muodostavat syötteen myöhemmille kerroksille ja joissa opittavat parametrit voivat hyödyntää tokenien välisiä yhteyksiä yhteisen esitystilan kautta.

Upotusvektorit ovat myös yhteensopivia erikoistokenien kanssa. Esimerkiksi aloitusmerkki $\langle s \rangle$, lopetusmerkki $\langle /s \rangle$ ja tuntematon token $\langle unk \rangle$ voidaan sisällyttää laajennettuun sanastoon jolloin niille voidaan oppia omat upotuksensa samalla tavalla

kuin muillekin tokeneille. Näin myös sekvenssien rakenteelliset elementit voidaan käsitellä yhtenäisesti muun sanaston kanssa.

Upotusvektorit muodostavat siten luonnollisen jatkeen one-hot-esitykselle. Siinä missä one-hot kuvaa tokenin pelkkänä identiteettinä, upotusvektori tarjoaa tiiviin ja optimoitavan esityksen, joka soveltuu tehokkaasti neuroverkkopohjaisiin kielimalleihin.

3.2 Upotusvektorien oppiminen Word2Vec-menetelmällä

Upotusvektorit eivät tavallisesti ole ennalta määrättyjä, vaan ne estimoidaan aineistosta optimointimenetelmän avulla. Tässä suhteessa yksi keskeisimmistä ja vaikutusvaltaisimmista menetelmistä on Word2Vec, jonka Mikolov ym. esittelevät työssään *Efficient Estimation of Word Representations in Vector Space* [23]. Menetelmän keskeinen tieteellinen kontribuutio ei rajoitu siihen, että sanoille opitaan tiheitä vektoriesityksiä, sillä vastaavia ajatuksia oli esitetty neuroverkkopohjaisessa kielimallinnuksessa jo aiemmin. Word2Vecin varsinainen merkitys liittyy ennen kaikkea siihen, että se osoitti tällaisten representaatioiden oppimisen olevan mahdollista laskennallisesti erittäin tehokkaalla tavalla myös hyvin suurissa korpuksissa. Lisäksi se toi esiin, että kontekstuaaliseen ennustamiseen perustuva oppiminen voi johtaa vektoriavaruuksiin, joissa ilmenee kielellisesti mielekkäitä semanttisia ja syntaktisia säännönmukaisuuksia.

Word2Vec perustuu distributionaaliseen hypoteesiin, jonka mukaan sanan merkitys voidaan päätellä sen käyttöympäristöjen perusteella. Toisin sanoen sanat, jotka esiintyvät samankaltaisissa konteksteissa, saavat oppimisprosessissa samankaltaiset vektoriesitykset. Tämä periaate on laajasti koko modernin kieliteknologian taustalla, mutta Word2Vec teki siitä käytännöllisen ja skaalautuvan laskennallisen menetelmän. Menetelmä osoitti, että diskreettien symbolien semanttista rakennetta voidaan mallintaa jatkuvassa vektoriavaruudessa pelkästään paikallisiin konteksteihin perustuvan ennustetehtävän avulla ilman käsin suunniteltuja kielellisiä piirteitä.

Olkon korpus $\mathcal{W} = (w_1, w_2, \dots, w_{|\mathcal{W}|})$, missä $w_t \in \mathcal{V}$ on sanaston $\mathcal{V}$ sana ajanhetkellä t . Word2Vec-malleissa tarkastellaan tyypillisesti kunkin sanan ympärille määritettyä paikallista kontekstia, jonka laajuus määräytyy ikkunaparametrilla $c \in \mathbb{N}$. Tällöin ajanhetken t kontekstiksi voidaan määritellä

$$C_t = \{w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c}\},$$

olettaen, että indeksit pysyvät korpuksen rajojen sisällä. Mikolov ym. [23] tarkastelevat kahta vaihtoehtoista perusarkkitehtuuria: jatkuvien sanojen pussimallia (*continuous bag-of-words*, CBOW) ja Skip-gram-mallia.

CBOW-mallissa pyritään ennustamaan tarkasteltava keskussana sen kontekstisanojen perusteella. Tavoitteena on siten approksimoida todennäköisyyttä

$$p(w_t \mid w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c}).$$

Kontekstin sanat projisoidaan ensin vektoriesityksiksi, minkä jälkeen nämä esitykset yhdistetään tavallisesti summaamalla tai keskiarvoistamalla. Muodostettua kontekstivektoria käytetään tämän jälkeen kohdesanan ennustamiseen koko sanaston yli.

CBOW-mallin etuna on sen laskennallinen tehokkuus erityisesti suurilla aineistoilla, koska useiden kontekstisanojen informaatio voidaan tiivistää yhdeksi esitykseksi ennen ennustevaihetta [23].

Skip-gram-mallissa ennustesuunta on päinvastainen. Siinä tavoitteena on ennustaa keskussanan avulla sitä ympäröivät kontekstisanat. Tällöin maksimoidaan todennäköisyydet

$$p(w_{t+j} \mid w_t), \quad -c \leq j \leq c, j \neq 0.$$

Koko korpuksen tasolla vastaava tavoitefunktio voidaan kirjoittaa muodossa

$$\sum_{t=1}^{|\mathcal{W}|} \sum_{\substack{-c \leq j \leq c \\ j \neq 0}} \log p(w_{t+j} \mid w_t).$$

Skip-gram-malli osoittautuu erityisen käyttökelpoiseksi silloin, kun tavoitteena on oppia informatiivisia esityksiä myös harvinaisemmille sanoille, sillä kukin havaittu sana toimii useiden kontekstisanojen ennustamisen lähtökohtana [23].

Olkoon w_I syötesana ja w_O kohdesana. Jos syötesanaa vastaava rivivektoriesitys on $v_{w_I} \in \mathbb{R}^{1 \times d}$ ja kohdesanaa vastaava ulostulorivivektori on $u_{w_O} \in \mathbb{R}^{1 \times d}$, voidaan todennäköisyys $p(w_O \mid w_I)$ esittää softmax-funktion avulla muodossa

$$p(w_O \mid w_I) = \frac{\exp(v_{w_I} u_{w_O}^\top)}{\sum_{w \in \mathcal{V}} \exp(v_{w_I} u_w^\top)}.$$

Tässä nimittäjän summa ulottuu koko sanastoon, mikä tekee täsmällisestä optimoinnista laskennallisesti raskasta suurilla sanastoilla. Juuri tämän ongelman ratkaiseminen on yksi Word2Vecin keskeisistä ansioista. Menetelmä teki näkyväksi sen, että suuren sanaston ulostuloprojektio muodostaa vektoriesitysten oppimisessa keskeisen laskennallisen pullonkaulan, ja esitti tähän käytännöllisiä approksimaatioita. Näistä erityisen merkittävä on negatiivinen otanta (*negative sampling*), jossa koko sanaston yli määritellyn softmax-todennäköisyyden optimoinnin sijasta ratkaistaan joukko binäärisiä luokittelutehtäviä.

Olkoon (w_I, w_O) havaittu syöte–kohdesana-pari, jossa w_O esiintyy sanan w_I kontekstissa. Negatiivisessa otannassa valitaan lisäksi K negatiivista sanaa $w_1^-, \dots, w_K^-$ jakaumasta $P_n(w)$, joka on tyypillisesti jokin sanaston sanoille määritelty näytteenottojakauma. Tällöin mallin tavoitteena ei ole enää optimoida todennäköisyyttä $p(w_O \mid w_I)$ koko sanaston yli, vaan maksimoida havaittua paria vastaava binäärinen logistinen tavoite

$$\log \sigma(v_{w_I} u_{w_O}^\top) + \sum_{k=1}^K \log \sigma(-v_{w_I} u_{w_k^-}^\top),$$

missä $\sigma(x) = \frac{1}{1+e^{-x}}$ on logistinen sigmoidifunktio. Ensimmäinen termi kasvattaa havaittuun positiiviseen sana–konteksti-pariin liittyvän pistetulon $v_{w_I} u_{w_O}^\top$ arvoa, kun taas jälkimmäinen termi pienentää satunnaisesti valittujen negatiivisten sanojen $v_{w_I} u_{w_k^-}^\top$ arvoja. Näin malli oppii erottamaan aidot kontekstisanat satunnaisista sanoista ilman, että jokaisella oppimisaskeleella tarvitsisi normalisoida jakaumaa koko sanaston yli.

Koko aineiston yli tavoite voidaan esittää odotusarvomuodossa

$$\sum_{(w_I, w_O) \in D} \left[\log \sigma(v_{w_I} u_{w_O}^\top) + \sum_{k=1}^K \mathbb{E}_{w_k^- \sim P_n} \log \sigma(-v_{w_I} u_{w_k^-}^\top) \right],$$

missä D on havaittujen syöte–konteksti-parien joukko ja K on negatiivisten otantojen määrä. Tällä tavoin alkuperäinen moniluokkainen softmax-optimointi approksimoidaan joukolla logistisia regressiotehtäviä, joissa erotellaan positiiviset havaintoparit negatiivisesti näytteistetyistä pareista. Laskennallinen etu syntyy siitä, että kunkin havaintoparin kohdalla päivitetään vain positiivista kohdesanaa ja K :ta negatiivista sanaa vastaavat parametrit koko sanaston kattavan päivityksen sijasta.

Word2Vec-mallissa opitaan kaksi parametrimatriisia: syötevektorit ja ulostulovektorit. Jokaiselle sanaston sanalle $v_j \in V$ assosioituu siten ainakin yksi tiheä vektoriesitys. Oppimisen jälkeen sanan lopullisena esityksenä käytetään tavallisesti syötevektoria tai vaihtoehtoisesti syöte- ja ulostulovektoreiden yhdistelmää. Näin saadut upotukset heijastavat korpuksessa havaittuja yhteisesiintymisrakenteita siten, että samankaltaisissa käyttöympäristöissä esiintyvät sanat sijoittuvat tyypillisesti lähelle toisiaan upotustilassa.

Mikolov ym. [23] osoittavat lisäksi, että Word2Vecin oppimat vektoriesitykset eivät ainoastaan ryhmittele semanttisesti läheisiä sanoja, vaan niihin syntyy myös lineaarisia relaatiota vastaavia säännönmukaisuuksia. Sanavektorien erotukset voivat vastata esimerkiksi sukupuoleen, lukuun tai taivutusmuotoon liittyviä semanttisia ja syntaktisia eroja, mikä ilmenee analogiatehtävissä. Tämä havainto on menetelmän kannalta erityisen merkittävä, sillä se viittaa siihen, että kontekstuaaliseen ennustamiseen perustuva optimointi tuottaa emergentisti sellaisia vektoriavaruuden rakenteita, jotka eivät ole eksplisiittisesti koodattuja malliin, mutta jotka vastaavat kielellisesti tulkittavia suhteita. Näin Word2Vec osoitti, että jakaumallinen oppiminen ei tuota ainoastaan samankaltaisuusmittaa, vaan voi synnyttää myös abstraktimpia relaatioita mallintavia geometrisia rakenteita.

Vaikka Word2Veciä ei nykyisissä suurissa kielimalleissa tyypillisesti käytetä sellaisenaan erillisenä menetelmänä, sen vaikutus näkyy edelleen usealla tasolla. Ensinnäkin se vakiinnutti distributionaalisen hypoteesin neuroverkkopohjaisen kielimallinuksen keskeiseksi oppimisperiaatteeksi. Toiseksi se toi esiin suuren sanaston ulostuloprojektion laskennallisen haasteen sekä popularisoi tehokkaita approksimointimenetelmiä, kuten negatiivisen otannan. Kolmanneksi se osoitti vakuuttavasti, että yksinkertainenkin kontekstuaalinen ennustetehtävä voi johtaa vektoriavaruuksiin, joissa semanttiset ja syntaktiset suhteet ilmenevät lineaarisina rakenteina. Näistä syistä Word2Vecia voidaan pitää tärkeänä siirtymävaiheena klassisten jakaumallisten sanamallien ja myöhempien syvien neuroverkkojen, mukaan lukien nykyaikaiset suuret kielimallit, välillä.

3.3 N-gram-malli optimointiongelmana

Luvussa 2.2 n -gram-malli esitettiin lukumääriin ja niiden normalisointiin perustuva frekvenssimallina. Sama malli voidaan kuitenkin tulkita myös suurimman uskottavuuden estimointina, mikä tekee yhteyden myöhempiin parametrisiin kielimalleihin näkyväksi.

Olkoon mallin parametreina jokaiselle kontekstille $c = (w_{t-n+1}, \dots, w_{t-1})$ määritelty todennäköisyysjakauma

$$\theta_c = (\theta_{c,1}, \theta_{c,2}, \dots, \theta_{c,|\mathcal{V}|}), \quad \sum_{j=1}^{|\mathcal{V}|} \theta_{c,j} = 1, \quad \theta_{c,j} \geq 0,$$

missä $\theta_{c,j} = P_\theta(w_t = v_j \mid c)$. Markov-oletuksen nojalla havaintoaineiston $\mathcal{W} = (w_1, \dots, w_{|\mathcal{W}|})$ uskottavuus voidaan kirjoittaa muodossa

$$P(\mathcal{W}; \theta) \approx \prod_{t=1}^{|\mathcal{W}|} P_\theta(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

Kun aineisto ryhmitellään kontekstin c ja sitä seuraavan tokenin v_j mukaan, saadaan

$$P(\mathcal{W}; \theta) = \prod_c \prod_{j=1}^{|\mathcal{V}|} \theta_{c,j}^{C_{c,j}},$$

missä $C_{c,j}$ on niiden havaintojen lukumäärä, joissa kontekstia c seuraa tokeni v_j .

Koneoppimisessa optimointi esitetään tavallisesti minimointitehtävänä, joten määritellään häviöksi negatiivinen log-uskottavuus

$$J(\theta; \mathcal{W}) = -\log P(\mathcal{W}; \theta) = -\sum_c \sum_{j=1}^{|\mathcal{V}|} C_{c,j} \log \theta_{c,j}.$$

Koska parametrit θ_c esiintyvät vain omaa kontekstiaan vastaavissa termeissä, optimointiongelma hajoaa erillisiksi kontekstikohtaisiksi osatehtäviksi:

$$\min_{\theta_c} -\sum_{j=1}^{|\mathcal{V}|} C_{c,j} \log \theta_{c,j} \quad \text{s.e.} \quad \sum_{j=1}^{|\mathcal{V}|} \theta_{c,j} = 1.$$

Tämä voidaan ratkaista Lagrangen menetelmällä. Määritellään

$$\mathcal{L}_{\text{Lag}}(\theta_c, \lambda) = \sum_{j=1}^{|\mathcal{V}|} C_{c,j} \log \theta_{c,j} + \lambda \left(1 - \sum_{j=1}^{|\mathcal{V}|} \theta_{c,j} \right).$$

Derivoimalla $\theta_{c,j}$:n suhteen saadaan

$$\frac{\partial \mathcal{L}_{\text{Lag}}}{\partial \theta_{c,j}} = \frac{C_{c,j}}{\theta_{c,j}} - \lambda = 0,$$

mistä seuraa

$$\theta_{c,j} = \frac{C_{c,j}}{\lambda}.$$

Kun tämä sijoitetaan normalisaatioehtoon

$$\sum_{j=1}^{|\mathcal{V}|} \theta_{c,j} = 1,$$

saadaan

$$\lambda = \sum_{k=1}^{|\mathcal{V}|} C_{c,k},$$

ja siten

$$\theta_{c,j} = \frac{C_{c,j}}{\sum_{k=1}^{|\mathcal{V}|} C_{c,k}}.$$

Tulos osoittaa, että suurimman uskottavuuden estimaatti on täsmälleen sama kuin luvussa 2.2 määritelty rivikohtaisesti normalisoitu frekvenssimatriisi F . Toisin sanoen klassinen frekvenssipohjainen n -gram-malli voidaan tulkita parametrisen mallin maksimiuskottavuusratkaisuna.

Tämä näkökulma tekee samalla näkyväksi mallin keskeisen rajoitteen: jokaiselle mahdolliselle kontekstille tarvitaan oma erillinen todennäköisyysjakaumansa. Kuten luvussa 2.2 nähtiin, kontekstin pituuden kasvaessa parametrien määrä kasvaa nopeasti, jolloin malli muuttuu harvaksi ja yleistää heikosti harvinaisiin tai havaitsemattomiin konteksteihin. Tästä syystä on luontevaa siirtyä malleihin, joissa ehdolliset todennäköisyydet opitaan jaetun parametrisaation avulla jatkuvassa avaruudessa sen sijaan, että ne talletettaisiin erillisinä taulukoituina jakaumina.

3.4 Monikerroksinen perseptroni seuraavan tokenin ennustajana

Seuraavan tokenin ehdollista jakaumaa voidaan approksimoida parametrisoidulla neuroverkolla. Yksi yksinkertainen ja historiallisesti keskeinen ratkaisu tähän on monikerroksinen perseptroni (engl. *multilayer perceptron*, MLP).

MLP on eteenpäinsyöttävä keinotekoinen neuroverkko, joka koostuu syötekerroksesta, yhdestä tai useammasta piilokerroksesta sekä ulostulokerroksesta. Sen historialliset juuret ovat Frank Rosenblattin perseptronitutkimuksessa [31][32]. Näissä töissä esitettiin varhaisia perseptronimalleja ja kerroksellisia rakenteita, mutta tehokas yleiskäyttöinen menetelmä kaikkien kerrosten parametrien oppimiseen puuttui vielä.

Neuroverkkojen varhaista kehitystä hidasti merkittävästi Marvin Minskyn ja Seymour Papertin teos, jossa analysoitiin yksikerroksisten perseptronien rajoitteita [24]. Teos osoitti, että yksinkertainen perseptroni ei voi ratkaista tiettyjä lineaarisesti erottamattomia ongelmia, kuten XOR-tehtävää. Vaikka kritiikki kohdistui ensisijaisesti yksikerroksisiin malleihin, sen vaikutus ulottui laajemmin neuroverkko-tutkimukseen ja osaltaan vähensi kiinnostusta monikerroksisiin verkkoihin useiden vuosien ajaksi.

Monikerroksisten verkkojen tehokkaan opettamisen kannalta keskeinen teoreettinen lähtökohta syntyi Seppo Linnainmaan työssä, jossa esitettiin käänteisen suunnan automaattinen derivointi menetelmä [20]. Paul Werbos puolestaan sovelsi tätä ajatusta neuroverkkoihin väitöskirjassaan ja myöhemmin artikkelissaan, jossa termi vastavirta-algoritmi (engl. *backpropagation*) vakiintui [45][44].

Monikerroksisten eteenpäinsyöttävien verkkojen läpimurto tapahtui kuitenkin vasta, kun David Rumelhart, Geoffrey Hinton ja Ronald Williamsin teos popularisoi

vastavirta-algoritmin[33]. Tämä työ teki MLP:stä käytännöllisen gradienttipohjaisesti opetettavan funktion approksimaattorin ja loi perustan myöhemmälle syväoppimiselle.

Matemaattisesti MLP voidaan ymmärtää funktion approksimaattorina, joka muodostaa kuvauksen syöteavaruudesta tulosteavaruuteen. Tämä kuvaus toteutetaan yhdistettynä funktiona, joka koostuu peräkkäisistä affineista muunnoksista ja niitä seuraavista epälineaarisista aktivaatiofunktioista. Tämän ansiosta MLP kykenee mallintamaan sellaisia syöteen ja tulosteen välisiä riippuvuuksia, joita lineaarinen malli ei voi esittää. Mallin toimintaa ohjaavat opittavat parametrit, tyypillisesti painomatriisit ja bias-vektorit, joiden arvot estimoidaan datasta minimoimalla valittua kohdefunktiota numeerisilla optimointimenetelmillä, kuten gradienttimenetelyllä [15].

Olkkoon c tarkasteltava konteksti ja $x(c) \in \mathbb{R}^{1 \times d_0}$ sitä vastaava syöterivivektori. Monikerroksinen perceptroni muuntaa tämän syöteen vaiheittain piirre-esitykseksi soveltamalla peräkkäisiä affiinisia muunnoksia ja epälineaarisia aktivaatioita. Jos mallissa on L piilokerrosta, voidaan sen laskenta esittää muodossa

$$\begin{aligned} h^{(0)} &= x(c), \\ a^{(i)} &= h^{(i-1)}W^{(i)} + b^{(i)}, & i = 1, \dots, L, \\ h^{(i)} &= \phi^{(i)}(a^{(i)}), & i = 1, \dots, L, \end{aligned}$$

missä $W^{(i)} \in \mathbb{R}^{d_{i-1} \times d_i}$ on kerroksen i painomatriisi, $b^{(i)} \in \mathbb{R}^{1 \times d_i}$ sen bias-rivivektori, $a^{(i)} \in \mathbb{R}^{1 \times d_i}$ affiinisien muunnosten tulos ja $h^{(i)} \in \mathbb{R}^{1 \times d_i}$ kerroksen i piiloesitys rivivektorina. Funktio $\phi^{(i)}$ on kerroksen epälineaarinen aktivaatiofunktio.

Viimeisestä piilokerroksesta muodostetaan ulostulon normalisoimattomat pistemäärät eli logitit rivivektorina

$$\ell = h^{(L)}W^{(\text{out})} + b^{(\text{out})},$$

missä $W^{(\text{out})} \in \mathbb{R}^{d_L \times |\mathcal{V}|}$, $b^{(\text{out})} \in \mathbb{R}^{1 \times |\mathcal{V}|}$ ja $\ell \in \mathbb{R}^{1 \times |\mathcal{V}|}$. Vektorin ℓ komponentti ℓ_j vastaa sanaston tokenin v_j normalisoimatonta pistemäärää. Näistä muodostetaan ehdollinen todennäköisyysjakauma softmax-funktiolla:

$$p_\theta(w_t = v_j \mid c) = \frac{\exp(\ell_j)}{\sum_{k=1}^{|\mathcal{V}|} \exp(\ell_k)}, \quad j = 1, \dots, |\mathcal{V}|.$$

MLP muuntaa kontekstin ensin yhden tai usean kerroksen kautta abstraktiksi piirre-esitykseksi, jonka perusteella se laskee jokaiselle sanaston tokenille pistemäärän ja normalisoi nämä pistemäärät todennäköisyysjakaumaksi.

Mallin oppiminen perustuu samaan negatiiviseen log-uskottavuuden minimointiin kuin n -gram-mallissa:

$$J(\theta) = -\frac{1}{|\mathcal{W}|} \sum_{t=1}^{|\mathcal{W}|} \log p_\theta(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

Erona perinteiseen frekvenssipohjaiseen n -gram-malliin, joka estimoi jokaiselle mahdolliselle diskreetille kontekstille oman erillisen parametrivektorinsa, MLP approksimoi todennäköisyyksiä jatkuvassa avaruudessa. Koska erilliset tokenit projisoidaan matalaulotteisiksi tiheiksi upotusvektoreiksi $v \in \mathbb{R}^{1 \times d}$, malli oppii esittämään sanojen semanttisia ja syntaktisia samankaltaisuuksia vektoriesitysten avulla.

MLP:n approksimoima funktio on jatkuva ja tyypillisesti sileä siinä mielessä, että pienet muutokset syöteavaruudessa voivat johtaa samankaltaisiin ulostulojakaumiin. Tästä syystä malli kykenee yleistämään myös harvinaisiin tai aiemmin havaitsemattomiin konteksteihin hyödyntämällä sellaisten koulutusdatassa esiintyneiden kontekstien rakennetta, joiden esitykset sijaitsevat lähellä niitä yhteisessä avaruudessa [3]. Tämä lieventää klassisia n -gram-malleja vaivaavaa nollatodennäköisyyksien ongelmaa ilman erillisiä tasoitusmenetelmiä.

Frekvenssipohjaista mallia koskevat tekniset rajoitteet, kuten eksponentiaalinen muistitarpeen kasvu ja lukumatriisin harvuus, voidaan siten osittain ratkaista MLP-mallin avulla. MLP-rakenteessa opittavat painomatriisit ja bias-vektorit jakautuvat kaikkien kontekstien kesken. Siten jokainen harjoitusnäyte päivittää koko neuroverkon parametreja, mikä parantaa myös sellaisten lauseiden ennustamista, joita ei ole esiintynyt opetusdatassa, mutta jotka jakavat samankaltaisia piirteitä opittujen esitysten kautta. Tämä tekee MLP-pohjaisesta n -gram-mallista huomattavasti joustavamman, muistikapasiteetiltaan tehokkaamman ja yleistyskykyisemmän kuin perinteinen frekvenssiperusteinen taulukkolukuesitys [3, 15, 17].

4 Gradienttimenetelmä

Gradienttimenetelmä ja sen stokastinen muunnelmä ovat ensimmäisen kertaluvun optimointimenetelmiä, joissa kohdefunktion gradienttia hyödynnetään parametrivektorin päivittämisessä. Tarkastellaan minimointitehtävää

$$\min_{\theta \in \mathbb{R}^d} J(\theta),$$

missä $J : \mathbb{R}^d \rightarrow \mathbb{R}$ on jatkuvasti derivoituva kohdefunktio ja $\theta \in \mathbb{R}^d$ sen parametrivektori. Gradienttimenetelmän perusajatus perustuu kohdefunktion ensimmäisen kertaluvun paikalliseen approksimaatioon. Olkoon $\theta_t \in \mathbb{R}^d$ ajanhetken t parametrivektori ja $v \in \mathbb{R}^d$ pieni siirtymävektori. Taylorin lauseen nojalla saadaan

$$J(\theta_t + v) = J(\theta_t) + \langle \nabla J(\theta_t), v \rangle + o(\|v\|_2), \quad \text{kun } \|v\|_2 \rightarrow 0.$$

Tästä seuraa, että funktion arvon paikallista muutosta approksimoi ensisijaisesti lineaarinen termi $\langle \nabla J(\theta_t), v \rangle$. Rajoittamalla siirtymän pituutta ehdolla $\|v\|_2 \leq \varepsilon$ saadaan minimointitehtävä

$$\min_{\|v\|_2 \leq \varepsilon} \langle \nabla J(\theta_t), v \rangle.$$

Olettamalla, että $\nabla J(\theta_t) \neq 0$, Cauchy–Schwarzin epäyhtälöstä seuraa arvio

$$\langle \nabla J(\theta_t), v \rangle \geq -\|\nabla J(\theta_t)\|_2 \|v\|_2 \geq -\varepsilon \|\nabla J(\theta_t)\|_2.$$

Yhtäsuuruus toteutuu, kun vektori v on vastakkaissuuntainen gradienttiin nähden. Näin ollen lineaarisen approksimaation kannalta optimaalinen siirtymä on

$$v^* = -\frac{\varepsilon}{\|\nabla J(\theta_t)\|_2} \nabla J(\theta_t).$$

Optimaalinen siirtymä v^* osoittaa, että funktion arvo pienenee paikallisesti nopeimmin negatiivisen gradientin suuntaan. Uusi parametriarvo saadaan asettamalla $\theta_{t+1} = \theta_t + v^*$. Käytännössä normalisointitermi $\varepsilon/\|\nabla J(\theta_t)\|_2$ korvataan usein suoraan oppimisnopeudella $\eta > 0$ (engl. *learning rate*), jolloin askelpituus skaalautuu luonnostaan gradientin suuruuden mukaan. Tämä johtaa gradienttimenetelmän iteraatiiviseen päivityssääntöön

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t).$$

Koneoppimisessa kohdefunktio muodostetaan tyypillisesti empiirisenä riskinä

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\theta; z_i),$$

missä $z_1, \dots, z_N$ ovat opetusdatan näytteitä ja $\ell(\theta; z_i)$ on havaintoon z_i liittyvä häviöfunktio. Tällöin täysi gradientti on

$$\nabla J(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla \ell(\theta; z_i).$$

Kielimallien opetuksessa käytettävät aineistot ovat usein erittäin suuria, minkä vuoksi täyden gradientin laskeminen jokaisella iteraatiolla on laskennallisesti kallista. Tämän vuoksi käytetään stokastista gradienttimenetelmää (engl. *stochastic gradient descent*, SGD), jossa täysi gradientti korvataan satunnaisesti poimitun mini-erän perusteella muodostetulla gradienttiestimaatilla [30, 4, 5].

Olkoon $B_t \subset \{1, \dots, N\}$ hetkellä t muodostettu satunnainen mini-erä, jonka koko on $|B_t| = b$. Tällöin gradienttiestimaatti g_t määritellään muodossa

$$g_t = \frac{1}{b} \sum_{i \in B_t} \nabla \ell(\theta_t; z_i),$$

ja SGD:n päivityssääntö on

$$\theta_{t+1} = \theta_t - \eta g_t.$$

Jos mini-erä valitaan hetkellä t tasaisesti kaikkien b -alkioisten indeksijoukkojen joukosta, gradienttiestimaatti on ehdollisen odotusarvon mielessä harhaton [5]

$$\mathbb{E}[g_t \mid \theta_t] = \nabla J(\theta_t).$$

Stokastinen gradientti vastaa siten keskimäärin täyttä gradienttia, vaikka yksittäisen päivitys sisältää satunnaisuudesta johtuvaa kohinaa. Tämä kohina estää yleensä kohdefunktion arvon monotonisen pienenemisen, mutta mini-erien käyttö lieventää laskennallisia haasteita [4, 15].

Sekä gradienttimenetelmän että SGD:n päivitykset perustuvat kohdefunktion ensimmäisen kertaluvun paikalliseen approksimaatioon. Tämä approksimaatio ei kuitenkaan sellaisenaan takaa kohdefunktion arvon pienenemistä. Tällaisen takuun johtamiseksi tarvitaan gradientin säännöllisyyttä koskevia lisäoletuksia.

Määritelmä 1. Jatkuvasti derivoituva funktio $J : \mathbb{R}^d \rightarrow \mathbb{R}$ on L -sileä, jos sen gradientti ∇J on L -Lipschitz, eli kaikilla $\theta, \theta' \in \mathbb{R}^d$ pätee

$$\|\nabla J(\theta) - \nabla J(\theta')\|_2 \leq L\|\theta - \theta'\|_2.$$

Olettaen, että kohdefunktio on L -sileä, sen muutokselle voidaan johtaa neliöllinen yläraja.

Lemma 1 (Neliöllinen yläraja). *Olkoon $J : \mathbb{R}^d \rightarrow \mathbb{R}$ L -sileä funktio. Tällöin kaikilla $\theta, h \in \mathbb{R}^d$ pätee*

$$J(\theta + h) \leq J(\theta) + \langle \nabla J(\theta), h \rangle + \frac{L}{2} \|h\|_2^2.$$

Todistus. Analyysin peruslauseesta seuraa, että

$$J(\theta + h) - J(\theta) = \int_0^1 \langle \nabla J(\theta + th), h \rangle dt.$$

Lisäämällä ja vähentämällä termi $\langle \nabla J(\theta), h \rangle$ integraalin sisällä saadaan

$$\begin{aligned} J(\theta + h) - J(\theta) &= \int_0^1 \langle \nabla J(\theta), h \rangle dt \\ &\quad + \int_0^1 \langle \nabla J(\theta + th) - \nabla J(\theta), h \rangle dt \\ &= \langle \nabla J(\theta), h \rangle + \int_0^1 \langle \nabla J(\theta + th) - \nabla J(\theta), h \rangle dt. \end{aligned}$$

Cauchy–Schwarzin epäyhtälön ja L -Lipschitz-oletuksen perusteella integraalissa oleva termi voidaan arvioida ylhäältä

$$\begin{aligned}\langle \nabla J(\theta + th) - \nabla J(\theta), h \rangle &\leq \|\nabla J(\theta + th) - \nabla J(\theta)\|_2 \|h\|_2 \\ &\leq Lt \|h\|_2^2.\end{aligned}$$

Integroimalla saadaan

$$\begin{aligned}\int_0^1 \langle \nabla J(\theta + th) - \nabla J(\theta), h \rangle dt &\leq \int_0^1 Lt \|h\|_2^2 dt \\ &= \frac{L}{2} \|h\|_2^2.\end{aligned}$$

Näin ollen

$$J(\theta + h) \leq J(\theta) + \langle \nabla J(\theta), h \rangle + \frac{L}{2} \|h\|_2^2.$$

□

Neliöllinen yläraja tarjoaa keskeisen työkalun optimointimenetelmien analysointiin. Soveltamalla sitä gradienttimenetelmän päivitysaskeleeseen, jossa parametreja siirretään negatiivisen gradientin suuntaan askelpituudella $\eta > 0$, saadaan taattua kohdefunktion arvon pieneneminen kullakin iteraatiolla. Tämä tulos tunnetaan laskeutumislemmanna.

Lemma 2 (Laskeutuslemma). *Olkoon $J : \mathbb{R}^d \rightarrow \mathbb{R}$ L -sileä funktio ja $0 < \eta \leq 1/L$. Tällöin gradienttimenetelmän päivityksellä*

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

pätee

$$J(\theta_{t+1}) \leq J(\theta_t) - \frac{\eta}{2} \|\nabla J(\theta_t)\|_2^2.$$

Todistus. Sovelletaan neliöllistä ylärajaa (lemma 1) sijoittamalla $\theta = \theta_t$ ja $h = -\eta \nabla J(\theta_t)$, jolloin $\theta + h = \theta_{t+1}$. Tällöin saadaan

$$\begin{aligned}J(\theta_{t+1}) &\leq J(\theta_t) + \langle \nabla J(\theta_t), -\eta \nabla J(\theta_t) \rangle + \frac{L}{2} \|\eta \nabla J(\theta_t)\|_2^2 \\ &= J(\theta_t) - \eta \|\nabla J(\theta_t)\|_2^2 + \frac{L\eta^2}{2} \|\nabla J(\theta_t)\|_2^2 \\ &= J(\theta_t) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla J(\theta_t)\|_2^2.\end{aligned}$$

Koska $0 < \eta \leq 1/L$, pätee $1 - \frac{L\eta}{2} \geq \frac{1}{2}$, mistä seuraa

$$J(\theta_{t+1}) \leq J(\theta_t) - \frac{\eta}{2} \|\nabla J(\theta_t)\|_2^2.$$

□

Lemma osoittaa, että kohdefunktion arvo pienenee jokaisessa ei-stationaarisessa pisteessä, kun oppimismen nopeus valitaan ehdon $0 < \eta \leq 1/L$ mukaisesti. Käytännössä oppimismen nopeuden valinta ei kuitenkaan ole suoraviivaista, sillä kohdefunktion kaa-revuus voi vaihdella merkittävästi eri parametrisuunnissa. Seuraavassa alaluvussa tarkastellaan tämän vaihtelun aiheuttamia haasteita optimoinnille.

4.1 Häiriöalttius ja toisen kertaluvun geometria

Stokastisen gradienttimenetelmän keskeinen haaste on, että kohdefunktion lokaali kaarevuus voi vaihdella huomattavasti eri parametrisuunnissa. Tätä ilmiötä kutsutaan numeeriseksi häiriöalttiudeksi (engl. *ill-conditioning*). Olkoon kohdefunktio J L -sileä ja kahdesti jatkuvasti derivoituva. Tällöin pisteen θ_t ympäristössä pätee toisen kertaluvun Taylorin approksimaatio

$$J(\theta_t + \Delta\theta) \approx J(\theta_t) + \langle \nabla J(\theta_t), \Delta\theta \rangle + \frac{1}{2} \langle \Delta\theta, \nabla^2 J(\theta_t) \Delta\theta \rangle.$$

Merkitään Hessin matriisia pisteessä θ_t symbolilla

$$H_t = \nabla^2 J(\theta_t).$$

Koska H_t on symmetrinen, sen ominaisarvot ovat reaalisia. Oletetaan tässä, että lokaali kvadraattinen malli on konvekksi eli $H_t \succeq 0$. Tällöin ominaisarvot voidaan järjestää muodossa

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0,$$

missä λ_1 vastaa suurinta ja λ_d pienintä paikallista kaarevuutta.

Funktion L -sileyden nojalla Hessin operaattorinormille pätee $\|H_t\|_2 \leq L$, jolloin $\lambda_1 \leq L$. Tämä rajoittaa oppimisnopeuden suuruutta, koska liian suuri askel voi johtaa divergenssiin tai voimakkaaseen oskillaatioon suurimman kaarevuuden suunnassa. Häiriöalttiuden vaikutusta suppenemiseen voidaan havainnollistaa tarkastelemalla stationaarisen pisteen θ^* ympäristöä, jossa $\nabla J(\theta^*) = 0$. Merkitsemällä $H^* = \nabla^2 J(\theta^*)$ ja virhevektoria $e_t = \theta_t - \theta^*$ saadaan lokaaliksi linearisoiduksi virhedynamiikaksi

$$e_{t+1} \approx (I - \eta H^*) e_t.$$

Kun virhevektori esitetään H^* :n ortonormaalissa ominaisvektorikannassa, kukin virhekomponentti supistuu kertoimella

$$e_{t+1}^{(k)} \approx (1 - \eta \lambda_k) e_t^{(k)}, \quad k = 1, \dots, d.$$

Yhtä oppimisnopeutta käytettäessä syntyy siten perustavanlaatuinen kompromissi. Jos η valitaan suureksi ($\eta \approx 1/\lambda_1$) nopean etenemisen saavuttamiseksi jyrkimmän kaarevuuden suunnassa, loivimmissa suunnissa pätee $1 - \eta \lambda_d \approx 1$. Tällöin virhe näissä suunnissa supistuu hyvin hitaasti. Ongelman vakavuutta kuvaa Hessin matriisin ehtoluku

$$\kappa = \frac{\lambda_1}{\lambda_d},$$

kun $\lambda_d > 0$. Suuri ehtoluku tarkoittaa, että gradienttimenetelmä etenee kapeassa laaksossa oskilloiden voimakkaan kaarevuuden suunnassa ja vain hitaasti laakson pohjan suuntaisesti [25].

Toisen kertaluvun optimointimenetelmät, kuten Newtonin menetelmä, hyödyn-
tävät paikallista kaarevuusinformaatiota skaalaamalla päivitysaskelen käänteisellä
Hessin matriisilla

$$\theta_{t+1} = \theta_t - \eta H_t^{-1} \nabla J(\theta_t).$$

Tällöin päivityssuunta mukautuu sekä eri suuntien kaarevuuseroihin että paramet-
rien välisiin riippuvuuksiin, joita Hessin matriisin ei-diagonaaliset alkiot kuvaavat.
Nykyaikaisissa neuroverkoissa ja kielimalleissa parametrien lukumäärä d on kui-
tenkin tyypillisesti miljoonia tai miljardeja. Täyden Hessin matriisin tallentaminen
vaatii $\mathcal{O}(d^2)$ muistia, ja sen kääntäminen vaatii $\mathcal{O}(d^3)$ laskutoimitusta. Tästä syys-
tä täyteen kaarevuusinformaatioon perustuvien menetelmien suora soveltaminen on
suuressa mittakaavassa yleensä epäkäytännöllistä.

4.2 Adaptiiviset optimointimenetelmät

Syväoppimisessa hyödynnetään tämän vuoksi laajasti ensimmäisen kertaluvun op-
timointimenetelmiä, jotka pyrkivät parantamaan gradienttimenetelmien käytännön
toimintaa pienellä laskennallisella lisäkustannuksella [15]. Keskeisiä menetelmäper-
heitä ovat *momentum*-menetelmät, jotka vaimentavat häiriöalttiuden aiheuttamaa
oskillointia suodattamalla päivityssuuntaa ajassa, sekä adaptiiviset optimointime-
netelmät [18].

Adaptiivisten menetelmien keskeinen periaate on muodostaa parametrikohtaisia
oppimisnopeuksia gradienttistorian perusteella. Menetelmät mukauttavat päivi-
tysten mittakaavaa erityisesti silloin, kun gradienttien suuruus vaihtelee huom-
tatavasti parametrikoordinaattien välillä. Syvissä neuroverkoissa eri kerrosten para-
metrit sekä esimerkiksi harvoin esiintyviä sanoja vastaavat upotusvektorit voivat
saada hyvin erisuuruisia gradientteja [12]. Yhden globaalin oppimisnopeuden käyt-
tö voi tällöin johtaa siihen, että suuria gradientteja saavat parametrit päivittyvät
liian voimakkaasti, kun taas pieniä gradientteja saavat parametrit etenevät hitaasti.

Adaptiiviset menetelmät tasapainottavat tätä vaihtelua skaalaamalla kunkin pa-
rametrin päivitystä sen aiemmin saamien gradienttien perusteella. Tämä voidaan
tulkita diagonaalisen esiehdoitukseksi. Jokaiselle parametrille määritetään oma
päivitysaskelen mittakaava ilman, että parametrien välisiä riippuvuuksia tarvitsee
käsitellä eksplisiittisesti [5, 12]. Tämä tekee menetelmistä laskennallisesti tehokkaita
ja erityisen soveltuvia suuriin, korkeaulotteisiin optimointiongelmiiin.

Yleinen adaptiivinen menetelmä voidaan esittää muodossa

$$\theta_{t+1} = \theta_t - \eta_t D_t g_t,$$

missä g_t on hetkellä t laskettu stokastinen gradientti, $\eta_t > 0$ on globaali oppimis-
nopeus ja $D_t \in \mathbb{R}^{d \times d}$ on positiivisesti definiitti diagonaalinen skaalausmatriisi, joka
muodostetaan aiempien gradienttien perusteella. Diagonaalisuuden vuoksi menetel-
mien muisti- ja aikavaativuus on vain $\mathcal{O}(d)$. Tämä mahdollistaa akselinsuuntaisten
mittakaavaerojen tehokkaan tasaamisen, vaikka se ei korjaakaan koordinaatiston
rotaatioista johtuvaa yleistä häiriöalttiutta.

Adaptiivisten menetelmien kehitys syväoppimisessa on rakentunut erityisesti Ada-
Gradin, RMSPropin ja Adamin ympärille. AdaGrad tuo esiin parametrikohtaisen op-
pimisnopeuden perusidean, RMSProp korjaa AdaGradin käytännöllistä heikkoutta

ja Adam yhdistää adaptiivisen skaalaamisen sekä momentumin. Modernissa neuroverkkojen ja erityisesti transformer-arkkitehtuurien opetuksessa myös AdamW on saavuttanut vakiintuneen aseman [12, 42, 18, 22, 43].

Momentum. Ennen puhtaasti adaptiivisia menetelmiä stokastisen gradienttime-
netelmän keskeisimmäksi laajennukseksi vakiintui momentum- eli liikemäärämenetelmä [26, 41]. Menetelmän tarkoituksena on nopeuttaa optimointia loivilla alueilla sekä vaimentaa oskillaatiota suuntiin, joissa kohdefunktion kaarevuus on suurta. Tämä saavutetaan lisäämällä päivitykseen liukuma aiemmista siirtymistä

$$v_t = \gamma v_{t-1} + \eta g_t,$$

missä $\gamma \in [0, 1)$ on momentum-kerroin. Parametripäivitys tehdään kertyneen nopeusvektorin v_t suunnassa

$$\theta_{t+1} = \theta_t - v_t.$$

Vaihtoehtoisesti momentum voidaan formuloida eksponentiaalisesti painotettuna gradienttien liukuvana keskiarvona $m_t = \beta m_{t-1} + (1 - \beta)g_t$, jolloin päivitys on muotoa $\theta_{t+1} = \theta_t - \eta m_t$. Tämä jälkimmäinen esitysmuoto toimii suoraan perustana Adamin ensimmäisen momentin laskennalle.

AdaGrad. AdaGrad (*adaptive gradient*) oli ensimmäisiä laajasti tunnettuja menetelmiä, joissa oppimisnopeus mukautetaan parametrikohteisesti gradienttistorian perusteella [12]. Menetelmässä ylläpidetään neliöityjen gradienttien kumulatiivista summaa

$$s_t = \sum_{\tau=1}^t g_{\tau}^2,$$

missä potenssi- ja neliöjuurioperaatiot sekä jakolasku tulkitaan komponenttikohtaisesti. Päivitys kirjoitetaan muodossa

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \varepsilon},$$

missä $\varepsilon > 0$ on pieni vakio numeerisen stabiiliuden varmistamiseksi.

AdaGradin keskeinen ominaisuus on, että parametreille, joiden gradientit ovat toistuvasti suuria, kertyy nopeasti suuri nimittäjä, jolloin niiden efektiivinen oppimisnopeus pienenee. Vastaavasti parametreille, joiden gradientit ovat harvinaisia tai pieniä, voivat saada suhteellisesti suurempia päivityksiä. Tämän vuoksi menetelmä soveltuu erityisesti tilanteisiin, joissa gradientit ovat harvoja. AdaGradin keskeinen heikkous on kuitenkin se, että kertymä s_t kasvaa monotonisesti. Tämän seurauksena askelpituudet voivat ajan myötä pienentyä liikaa ja optimointi hidastua olennaisesti.

RMSProp. RMSProp (*root mean square propagation*) kehitettiin käytännölliseksi korjaukseksi AdaGradin liian nopeasti pieneneville askelpituuksille. Menetelmää ei alun perin esitelty vertaisarvioidussa tieteellisessä artikkelissa, vaan se tuotiin

julkisuuteen Geoffrey Hintonin pitämällä Coursera-verkkokurssilla [42]. RMSPropin toimintaperiaate on sittemmin dokumentoitu laajasti alan perusteoksissa [15].

Sen sijaan, että neliöityjä gradientteja summattaisiin koko optimoinnin ajalta, RMSProp käyttää eksponentiaalisesti painotettua liukuvaa keskiarvoa

$$v_t = \rho v_{t-1} + (1 - \rho)g_t^2,$$

missä $\rho \in (0, 1)$ on muistiparametri. Päivityssääntö on

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{v_t} + \varepsilon}.$$

RMSProp säilyttää AdaGradin perusintuition parametrikohtaisesta skaalaamisesta, mutta käyttää pitkän aikavälin kertymän sijasta paikallisempaa estimaattia gradienttien mittakaavasta. Tämä estää oppimisnopeuden systemaattisen pienenty-
misen ja tekee menetelmästä käytännössä käyttökelpoisemman syvien neuroverkko-
jen opetuksessa [15].

Adam. Adam (*adaptive moment estimation*) yhdistää gradientin ensimmäisen mo-
mentin eli suunnatun liukuvan keskiarvon sekä neliöityjen gradienttien toisen mo-
mentin eli liukuvan keskiarvon [18]. Menetelmässä määritellään rekursiot

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1)g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2)g_t^2, \end{aligned}$$

missä $\beta_1, \beta_2 \in (0, 1)$ ovat muistiparametreja. Koska m_t ja v_t alustetaan tavallises-
ti nolllaksi, ne ovat alkuvaiheessa harhaisia kohti nolllaa. Tämän vuoksi käytetään
harhakorjattuja estimaatteja

$$\begin{aligned} \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t}. \end{aligned}$$

Päivitys saa tällöin muodon

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

Adamia voidaan tulkita menetelmäksi, jossa m_t vakauttaa päivityssuuntaa mo-
mentum-tyyppisesti, kun taas v_t skaalaa askelpituutta parametrikohtaisesti gra-
dienttien havaittuun mittakaavaan nähden. Tämä tekee menetelmästä käytännössä
usein tehokkaan ja suhteellisen robustin oppimisnopeuden valinnalle. Näistä syistä
Adamista on tullut yksi käytetyimmistä optimointimenetelmistä syväoppimisessa.

Adam ei kuitenkaan ole ongelmaton. On havaittu, että adaptiiviset menetelmät
eivät kaikissa tilanteissa tuota yhtä hyvää yleistymistä kuin huolellisesti viritetty
SGD momentumilla [46]. Lisäksi Adamin alkuperäinen algoritmi voi tietyillä gra-
dienttisekvensseillä epäonnistua konvergenssissa, minkä vuoksi sen konvergenssia-
nalyysia on myöhemmin tarkennettu esimerkiksi AMSGrad-menetelmän avulla [29].

Adaptiivisuutta ei siten tule tulkita yleispäteväksi ratkaisuna kaikkiin optimoin-
nin haasteisiin, vaan käytännöllisenä keinona muokata päivityksen geometriaa ja
vähentää oppimisnopeuden herkkyyttä.

AdamW. AdamW on Adamin muunnelma, jossa painojen kutistaminen (*weight decay*) erotetaan adaptiivisesta gradienttipäivityksestä [22]. Tavallisessa L_2 -reguloinnin ja Adamin yhdistelmässä regularisaatiotermi sekoittuu adaptiiviseen skaalaukseen tavalla, joka ei vastaa puhdasta painojen kutistamista. AdamW:ssä parametripäivitys kirjoitetaan tyypillisesti muodossa

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} - \eta \lambda \theta_t,$$

missä $\lambda \geq 0$ säätelee painojen kutistamisen voimakkuutta.

Tämä erotus on osoittautunut käytännössä tärkeäksi erityisesti syvien neuroverkkojen opetuksessa [22]. AdamW:tä pidetäänkin usein tarkoituksenmukaisempana valintana kuin alkuperäistä Adamia silloin, kun mallin opetuksessa käytetään eksplisiittistä painojen kutistamista. Transformer-arkkitehtuurien opetuksessa Adam-tyyppiset adaptiiviset menetelmät ovat yleisesti käytettyjä [43].

Lopuksi on syytä huomata, että hyvin suurissa kielimalleissa myös optimointimenetelmän muistikustannus muodostuu olennaiseksi. Adam ylläpitää jokaiselle parametrille ensimmäisen ja toisen momentin estimaatit, minkä vuoksi optimointitila voi kuluttaa huomattavan määrän muistia. Tämän vuoksi on kehitetty muistitehokkaampia adaptiivisia menetelmiä, kuten Adafactor, joissa pyritään säilyttämään adaptiivisen skaalaamisen keskeiset edut pienemmällä muistinkulutuksella [37].

Tässä tutkielmassa tarkastelun painopiste rajataan kuitenkin AdaGrad-, RMS-Prop- ja Adam-tyyppisiin menetelmiin, sillä ne muodostavat historiallisesti ja käsitteellisesti keskeisen perustan myöhemmille adaptiivisille optimointialgoritmeille.

5 Dekooderipohjainen Transformer-arkkitehtuuri suurille kielimalleille

Luvuissa 2 ja 3 kielimallinnusta tarkasteltiin seuraavan tokenin ehdollisen todennäköisyyden estimointina. Olkoon $\mathcal{V}$ tokenisanasto ja $\Delta(\mathcal{V})$ sanaston yli määriteltyjen todennäköisyysjakaumien joukko. Kielimalli voidaan tällöin kuvata funktiona, joka muuntaa aiemmin havaitun tokenikontekstin seuraavan tokenin todennäköisyysjakaumaksi.

n -gram-malli ennustaa seuraavaa tokenia enintään $n - 1$ sitä edeltävän tokenin perusteella. Se voidaan siten kuvata funktiona

$$f_{\text{ngm}}: \mathcal{V}^{n-1} \rightarrow \Delta(\mathcal{V}).$$

Myös luvussa 3 tarkasteltu MLP-pohjainen kielimalli käyttää kiinteän pituista kontekstia, vaikka sen parametrisoitu esitystapa mahdollistaa frekvenssipohjaista n -gram-mallia paremman yleistämisen.

Nykyaikaiset suuret kielimallit (engl. *large language models*, LLM) perustuvat samaan autoregressiiviseen ennustetehtävään, mutta niiden käsittelemän kontekstin pituus ei ole kiinteä. Olkoon L mallin suurin sallittu kontekstinpituus. Dekooderipohjainen Transformer-kielimalli voidaan kuvata funktiona

$$f_{\text{Tr}}: \bigcup_{m=0}^L \mathcal{V}^m \rightarrow \Delta(\mathcal{V}).$$

Malli voi siten muodostaa seuraavan tokenin jakauman millä tahansa enintään L tokenin pituisella kontekstilla. Käytännössä suurin kontekstinpituus on edelleen arkkitehtuurin ja laskentaresurssien rajoittama, mutta mallin parametrien ei tarvitse olla sidottuja yhteen tiettyyn kontekstin pituuteen.

Tässä luvussa tarkastellaan erityisesti dekodeeripohjaista Transformer-arkkitehtuuria, joka muodostaa useiden generatiivisten kielimallien, kuten GPT-malliperheen, matemaattisen perustan. Transformer esiteltiin alun perin Vaswanin ym. työssä *Attention Is All You Need* vuonna 2017 [43]. Alkuperäinen arkkitehtuuri oli konekäännökseen suunniteltu enkooderi-dekooderi-rakenne. Pian tämän jälkeen esitettiin, että pelkkään Transformer-dekooderiin perustuva autoregressiivinen kielimalli voidaan esikouluttaa suurilla tekstiaineistoilla ja hienosäätää useisiin kielenkäsittelytehtäviin [28].

5.1 Itsehuomio

Transformer-mallin keskeinen mekanismi on itsehuomio (engl. *self-attention*). Sen avulla tokenin esitys voidaan muodostaa koko syötekotekstin perusteella sen sijaan, että huomioitaisiin vain kiinteä määrä viimeisimpiä tokeneita. Kontekstin tokenien merkitys ei määräydy pelkän etäisyyden mukaan, vaan itsehuomio mahdollistaa kunkin tokenin kannalta olennaisen tiedon dynaamisen painottamisen yli koko sekvenssin.

Olkoon tarkasteltavan tokenijonon pituus T ja olkoon $h_i \in \mathbb{R}^{1 \times d}$ position i tokenia vastaava esitysvektori. Näistä muodostetaan syötematriisi

$$H = \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_T \end{bmatrix} \in \mathbb{R}^{T \times d}.$$

Itsehuomiomekanismi muodostaa syöte-esityksistä kolme erillistä lineaarista projektiota: kyselyn (engl. *query*), avaimen (engl. *key*) ja arvon (engl. *value*). Olkoot $W_Q, W_K \in \mathbb{R}^{d \times d_k}$ ja $W_V \in \mathbb{R}^{d \times d_v}$ opittavia painomatriiseja. Kysely-, avain- ja arvomatriisit lasketaan kertomalla syötematriisi näillä painoilla:

$$\begin{aligned} Q &= HW_Q \in \mathbb{R}^{T \times d_k}, \\ K &= HW_K \in \mathbb{R}^{T \times d_k}, \\ V &= HW_V \in \mathbb{R}^{T \times d_v}. \end{aligned}$$

Intuitiivisesti kysely kuvaa sitä, millaista tietoa kukin tokeni hakee, avain kuvaa tokenin tarjoamaa informaatiota muille, ja arvo sisältää varsinaisen sisällön, jota yhdistetään uusiin esityksiin.

Kyselyiden ja avainten välinen vastaavuus lasketaan pistetuloina, joista muodostuu skaalattu huomiopistematriisi

$$S = \frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{T \times T}.$$

Matriisin alkio S_{ij} mittaa position i kyselyn ja position j avaimen yhteensopivuutta. Skaalausta voidaan perustella tarkastelemalla kysely- ja avainvektoreita, joiden komponentit ovat riippumattomia, nollakeskiarvoisia ja yksikkövarianssisia. Näillä oletuksilla niiden pistetulon varianssi on d_k , joten jakaminen luvulla $\sqrt{d_k}$ palauttaa varianssin arvoon yksi. Vaikka oletukset eivät yleisesti toteudu täsmällisesti opituilla esityksillä, skaalaus hillitsee huomiopisteiden suuruuden kasvua dimension mukana. Tämä vähentää softmax-funktion saturoitumisen riskiä ja tukee vakaampaa gradienttipohjaista koulutusta [43].

Huomiopisteet normalisoidaan todennäköisyysjakaumiksi soveltamalla softmax-funktiota riveittäin, jolloin saadaan huomiopainomatriisi

$$A = \text{softmax}(S) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{T \times T},$$

missä kukin alkio $A_{ij} \in [0, 1]$ ilmaisee, kuinka suuri painoarvo position j arvolle annetaan position i uutta esitystä laskettaessa, ja riveille pätee $\sum_{j=1}^T A_{ij} = 1$.

Lopullinen itsehuomion tulos lasketaan arvomatriisin rivien painotettuna summana matriisitulon avulla

$$\text{Attention}(H) = AV = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \in \mathbb{R}^{T \times d_v}.$$

Tuloksena saadun matriisin kukin rivi i edustaa position i uutta kontekstualisoitua esitystä, johon on koottu tietoa kaikista sekvenssin tokeneista huomiopainojen A_{ij} mukaisesti.

Autoregressiivisissä kielimalleissa tavoitteena on mallintaa seuraavan tokenin todennäköisyyttä ja generoida tekstiä sekventiaalisesti, minkä vuoksi tokenin esitys ei saa hyödyntää tietoa tulevista tokeneista. Tätä varten itsehuomiota rajoitetaan *kausaalilla maskauksella*, joka estää informaation vuotamisen tulevaisuudesta asettamalla position i huomiopainot nolaksi kaikille positioille $j > i$. Tämä toteutetaan lisäämällä huomiopisteisiin ennen softmax-funktiota maskimatriisi $M \in (\mathbb{R} \cup \{-\infty\})^{T \times T}$, jonka alkiot määritellään muodossa

$$M_{ij} = \begin{cases} 0, & \text{kun } j \leq i, \\ -\infty, & \text{kun } j > i. \end{cases}$$

Kausaalinen itsehuomio lasketaan tällöin muodossa

$$\text{CausalAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V.$$

Koska $\exp(-\infty) = 0$, huomiopainomatriisista muodostuu alakolmiomatriisi. Tällöin kunkin tokenin esitys riippuu vain siitä itsestään ja sitä edeltävistä tokeneista, mikä säilyttää generointiprosessin kausaalisen rakenteen. Yksittäisen huomiopään tuloksella on hyödyllinen geometrinen tulkinta. Merkitään arvomatriisin V riviä j symbolilla $\mathbf{v}_j = V_{j,:} \in \mathbb{R}^{1 \times d_v}$. Koska huomiopainoille pätee $A_{ij} \geq 0$ ja $\sum_{j=1}^T A_{ij} = 1$, position i tulosrivivektori

$$\mathbf{o}_i = \sum_{j=1}^T A_{ij} \mathbf{v}_j \in \mathbb{R}^{1 \times d_v}$$

on arvovektorien konveksikombinaatio. Kausaalissa itsehuomiossa tulevien positioiden painot ovat nollia, joten tarkemmin

$$\mathbf{o}_i \in \text{conv}(\{\mathbf{v}_1, \dots, \mathbf{v}_i\}) = \left\{ \sum_{j=1}^i \alpha_j \mathbf{v}_j \mid \alpha_j \geq 0, \sum_{j=1}^i \alpha_j = 1 \right\}.$$

Tämä havainto koskee yhden huomiopään tulosta annetuille arvovektoreille. Se ei yksinään kuvaa koko Transformer-lohkon ilmaisukykyä, sillä arvovektorit ovat opittujen projektioiden tuloksia ja lohkon kuuluu myös muita muunnoksia.

Yhdessä huomiopäässä sama huomiopainotus kohdistuu arvovektorin kaikkiin koordinaatteihin. Monipäinen itsehuomio (engl. *multi-head attention*) mahdollistaa useiden erilaisten painotusten muodostamisen rinnakkain. Kullakin päällä on omat kysely-, avain- ja arvoprojektionsa. Eri päät voivat siten painottaa eri positioita ja käsitellä niiden informaatiota eri esitysavaruuksissa [43].

Olkoon päiden lukumäärä n_{heads} . Kukin pää $k \in \{1, \dots, n_{\text{heads}}\}$ suorittaa kausaalisen itsehuomion omilla opittavilla projektioiden:

$$\text{head}_k = \text{CausalAttention}\left(HW_k^Q, HW_k^K, HW_k^V\right),$$

missä $W_k^Q, W_k^K \in \mathbb{R}^{d \times d_k}$ ja $W_k^V \in \mathbb{R}^{d \times d_v}$. Kukin pää muodostaa näin oman huomiopainomatriisinsa ja omat painotetut summansa arvovektoreista. Päät voivat oppia

toisiaan täydentäviä riippuvuuksia. Esimerkiksi yksi pää saattaa keskittyä syntaktisiin rakenteisiin ja toinen laajempiin semanttisiin yhteyksiin. Todellisuudessa opitut suhteet eivät kuitenkaan jakaudu näin siististi, vaan ne ovat usein huomattavasti monimutkaisempia ja vaikeammin tulkittavia [9].

Lopuksi eri päiden tuottamat esitykset ketjutetaan ja yhdistetään lineaarisella projektiolla takaisin mallin ulottuvuuteen d :

$$\text{CausalMultiHead}(H) = [\text{head}_1 \parallel \text{head}_2 \parallel \dots \parallel \text{head}_{n_{\text{heads}}}] W^O,$$

missä $\parallel$ merkitsee sarakkeittaista ketjuttamista ja $W^O \in \mathbb{R}^{n_{\text{heads}} d_v \times d}$ on opittava parametrimatriisi.

Monipäisyyden etu on että eri päät voivat käyttää toisistaan poikkeavia huomiopainotuksia ja arvoprojektioita, joiden tuottama informaatio yhdistetään tulospojektiossa. Käytännössä päään dimensioiksi valitaan usein $d_k = d_v = d/n_{\text{heads}}$, jolloin monipäisen itsehuomion laskennallinen vaativuus vastaa karkeasti täyden dimension yksipäistä huomiota [43].

5.2 Positiokoodaus

Maskiton itsehuomio on permutaatioekvivariantti eli ilman erillistä positiotietoa se ei kykene erottelamaan tokenien järjestystä. Olkoon $P \in \mathbb{R}^{T \times T}$ permutaatiomatriisi, joka järjestää syötematriisin H rivit uudelleen. Koska kysely-, avain- ja arvomatriisit muodostetaan riveittäin samoilla lineaarisilla kuvauksilla, permutoidusta syötteestä saadaan $Q' = PQ$, $K' = PK$ ja $V' = PV$. Tällöin huomiopistematriisille pätee

$$\frac{Q'(K')^\top}{\sqrt{d_k}} = P \frac{QK^\top}{\sqrt{d_k}} P^\top.$$

Riveittäin sovellettava softmax-funktio säilyttää vastaavan permutaation, joten maskittomalle itsehuomiolle pätee

$$\text{Attention}(PH) = P \text{Attention}(H).$$

Syötteen tokenien permutointi siis ainoastaan permutoi tulosteet samalla tavalla. Maskiton itsehuomiomekanismi ei sellaisenaan sisällä tietoa siitä, missä järjestyksessä tokenit esiintyvät. Autoregressiivisessä mallissa kausaalinen maski rikkoo tämän ominaisuuden mielivaltaisten permutaatioiden suhteen, sillä se määrää, mitkä positiot ovat kunkin tokenin menneisyydessä. Kausaalinen maski tuo siten malliin järjestyksrakennetta, jonka avulla malli voi myös oppia hyödyntämään sijaintiin liittyvää tietoa. Maski ei kuitenkaan sisällytä kyselyn ja avaimen väliseen yhteensopivuuteen eksplisiittistä tietoa siitä, sijaitseeko avain esimerkiksi yhden vai usean tokenin päässä kyselystä. Erillinen positiokoodaus antaa mallille suoran mekanismin tokenien sijaintien tai niiden välisten positioerojen esittämiseen.

Positiotieto voidaan sisällyttää Transformer-arkkitehtuuriin usealla tavalla. Alkuperäisessä Transformerissa tokeniesityksiin lisätään absoluuttiset, sinimuotoiset positiokoodaukset [43]. Suhteellisissa menetelmissä huomiopisteitä puolestaan muokataan positoiden välisen etäisyyden perusteella. Monissa moderneissa dekodeeripohjaisissa kielimalleissa käytetään kiertävää positiokoodausta (engl. *rotary positional embedding*, RoPE), jossa positiotieto liitetään kysely- ja avainvektoreihin rotaatioiden avulla [40].

Olkoon yhden huomiopään kysely- ja avainvektorien dimensio d_k parillinen. RoPE jakaa vektorien koordinaatit $d_k/2$ kaksiulotteiseen pariin ja kiertää jokaista paria position mukaisella kulmalla. Koordinaattiparille $\ell \in \{0, \dots, d_k/2 - 1\}$ käytetään tavanomaisesti kulmataajuutta

$$\omega_\ell = 10000^{-2\ell/d_k}.$$

Kulmaa ϕ vastaava kaksiulotteinen rotaatiomatriisi on

$$R(\phi) = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}.$$

Position i koko rotaatiomatriisi muodostetaan näistä kaksiulotteisista rotaatioista lohkok diagonaalisenä matriisina

$$R_i = \text{diag} \left(R(i\omega_0), R(i\omega_1), \dots, R(i\omega_{d_k/2-1}) \right) \in \mathbb{R}^{d_k \times d_k}.$$

Olkoon $q_i = Q_{i,:} \in \mathbb{R}^{1 \times d_k}$ position i kyselyvektori ja $k_j = K_{j,:} \in \mathbb{R}^{1 \times d_k}$ position j avainvektori. RoPE korvaa nämä kierrettyillä rivivektoreilla

$$\tilde{q}_i = q_i R_i, \quad \tilde{k}_j = k_j R_j.$$

Arvovektoreihin rotaatiota ei tavallisesti sovelleta. Kierrettyjen kysely- ja avainvektorien pistetuloksi saadaan

$$\tilde{q}_i \tilde{k}_j^\top = q_i R_i (k_j R_j)^\top = q_i R_i R_j^\top k_j^\top.$$

Kaksiulotteisille rotaatiomatriiseille pätee $R(\alpha)R(\beta)^\top = R(\alpha - \beta)$. Sama ominaisuus pätee lohkoittain myös RoPE-rotaatioille, joten

$$R_i R_j^\top = R_{i-j}.$$

Siten RoPE-huomiopiste voidaan kirjoittaa muodossa

$$\tilde{q}_i \tilde{k}_j^\top = q_i R_{i-j} k_j^\top.$$

Vaikka kysely ja avain kierretään niiden absoluuttisten positioiden perusteella, rotaatioiden yhteisvaikutus pistetulossa riippuu ainoastaan positioiden suhteellisesta siirtymästä $i - j$. RoPE liittää näin suhteellisen positiotiedon suoraan kyselyiden ja avainten välisiin huomiopisteisiin ilman, että tokeniesityksiin täytyy lisätä erillisiä positiovektoreita.

Kun kierretyt kyselyt ja avaimet kootaan matriiseiksi $\tilde{Q}$ ja $\tilde{K}$, kausaalinen itsehuomio saa muodon

$$\text{CausalAttention}(\tilde{Q}, \tilde{K}, V) = \text{softmax} \left(\frac{\tilde{Q} \tilde{K}^\top}{\sqrt{d_k}} + M \right) V.$$

Koska R_i on ortogonaalinen matriisi, rotaatio säilyttää kysely- ja avainvektorien normit. Eri koordinaattipareille määritellyt kulmataajuudet määräävät, kuinka nopeasti rotaatiokulma muuttuu position funktiona. Suurilla taajuuksilla kulma muuttuu nopeasti position kasvaessa, kun taas pienillä taajuuksilla muutos on hitaampaa. Näin eri koordinaattiparit kuvaavat positioeroja eri mittakaavoilla. RoPE ei myöskään muuta itsehuomion asymptoottista laskennallista vaativuutta, sillä kysely- ja avainvektorien rotaatiot voidaan laskea tehokkaasti koordinaattipareittain.

5.3 Dekooderikerros

Yksittäinen dekooderikerros koostuu kausaalisesta monipäisestä itsehuomiosta ja positiokohtaisesta myötäkytkentäverkosta. Kumpaankin komponenttiin liitetään jäännöskytkentä (engl. *residual connection*) sekä normalisointi. Normalisointi auttaa hallitsemaan esitysten mittakaavaa, ja jäännöskytkennät helpottavat gradienttien kulkua syvässä verkossa. Yhdessä nämä rakenteet tukevat koulutuksen vakautta [16, 2, 43].

Laskennan vakauttamiseksi esitykset normalisoidaan ennen kuhunkin alilohkoon syöttämistä. Normalisointifunktio $\text{Norm}: \mathbb{R}^d \rightarrow \mathbb{R}^d$ sovelletaan itsenäisesti kunkin position esitysvektoriin. Yleinen valinta nykyaikaisissa malleissa on neliöllisen keskiarvon normalisointi (engl. *root mean square normalization*, RMSNorm) [50]. Tämä skaalaa vektorin sen komponenttien neliöllisen keskiarvon avulla

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \odot \gamma,$$

missä $\odot$ on Hadamard-tulo, $\gamma \in \mathbb{R}^d$ on opittava skaalausparametri ja

$$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2 + \epsilon},$$

missä pieni vakio $\epsilon > 0$ estää nollalla jakamisen. Matriisille $H \in \mathbb{R}^{T \times d}$ normalisointi $\text{Norm}(H)$ määritellään soveltamalla kuvausta riveittäin jokaiseen esitysvektoriin.

Itsehuomiokerrosta seuraa positiokohtainen myötäkytkentäverkko (engl. *feed-forward network*, FFN). Se vastaa luvussa 3 tarkasteltua monikerroksista perseptronin, jota sovelletaan itsenäisesti ja samoilla parametreilla kunkin position esitykseen.

Kaksi affinia muunnosta sisältävä FFN voidaan määritellä rivivektorille $x \in \mathbb{R}^{1 \times d}$ muodossa

$$\text{FFN}(x) = \phi(xW_1 + b_1)W_2 + b_2,$$

missä $W_1 \in \mathbb{R}^{d \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d}$, $b_1 \in \mathbb{R}^{1 \times d_{ff}}$, $b_2 \in \mathbb{R}^{1 \times d}$ ja ϕ on koordinaateittain sovellettava epälineaarinen aktivaatiofunktio. Sisäisen piiloesityksen dimensio d_{ff} valitaan tyypillisesti suuremmaksi kuin mallin dimensio d . Alkuperäisessä Transformer-arkkitehtuurissa käytettiin valintaa $d_{ff} = 4d$ [43].

Kun tokenien esitykset ovat matriisin $H \in \mathbb{R}^{T \times d}$ riveinä, sama laskenta kirjoitetaan muodossa

$$\text{FFN}(H) = \phi(HW_1 + \mathbf{1}b_1)W_2 + \mathbf{1}b_2,$$

missä $\mathbf{1} \in \mathbb{R}^{T \times 1}$ on sarakkeykkösvektori, jonka avulla siirtovektorit lisätään jokaiselle riville.

Koko dekooderilohko voidaan määritellä funktiona $\text{DecoderBlock}: \mathbb{R}^{T \times d} \rightarrow \mathbb{R}^{T \times d}$. Olkoon syöte esitysmatriisi $H \in \mathbb{R}^{T \times d}$. Lohkon laskenta etenee kahdessa vaiheessa:

$$H^{(1)} = H + \text{CausalMultiHead}(\text{Norm}_1(H)),$$

$$\text{DecoderBlock}(H) = H^{(1)} + \text{FFN}(\text{Norm}_2(H^{(1)})),$$

missä Norm_1 ja Norm_2 ovat toisistaan erillisiä normalisointeja omilla opittavilla parametreillaan. Normalisointi sijoitetaan ennen kumpaakin alilohkoa, joten rakenne noudattaa niin kutsuttua *pre-norm*-järjestystä [47].

Jäännöskytkennässä kunkin alilohkon syöte summataan suoraan sen tuottamaan muunnokseen. Tämä muodostaa gradientille suoran kulkureitin alilohkon ohi ja helpottaa syvien verkkojen optimointia [16]. Jäännöskytkennät eivät kuitenkaan yksin takaa koulutuksen vakautta, vaan myös normalisoinnin sijoittelu vaikuttaa koulutuksen dynamiikkaan [47].

Lohkon kahdella pääkomponentilla on toisiaan täydentävät tehtävät informaation käsittelyssä. Kausaalinen itsehuomio mahdollistaa sekvenssin eri positoiden välisen vuorovaikutuksen ja informaation reitittämisen tokenien välillä menneisyydestä nykyhetkeen [43]. Positiokohtainen myötäkytkentäverkko puolestaan vastaa kunkin position esitysvektorin riippumattomasta epälineaarista muunnoksesta ilman positoiden välistä tiedonsiirtoa [43]. Myötäkytkentäverkkoa voidaan lisäksi tulkita avain-arvomuistina, joka rikastaa tokenien esityksiä [14].

5.4 Dekooderikerroksista seuraavan tokenin jakaumaan

Dekooderilohko muuntaa tokenien esitysmatriisin uudeksi samankokoiseksi esitysmatriisiksi. Kokonainen kielimalli muodostetaan yhdistämällä tokenien upotus, useita peräkkäisiä dekodierilohkoja sekä seuraavan tokenin jakauman tuottava tuloskerros. Näin saadaan konkreettinen toteutus luvun alussa esitetylle kuvaukselle, joka muuntaa tokenikontekstin sanaston yli määritellyksi todennäköisyysjakaumaksi.

Olkoon syötteenä tokenijono $v_{1:T} = (v_1, \dots, v_T)$, missä $1 \leq T \leq L$, ja olkoon $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ opittava upotusmatriisi. Tokenin v_i one-hot-esitys on rivivektori $x(v_i) \in \mathbb{R}^{1 \times |\mathcal{V}|}$, ja sen upotus määritellään muodossa

$$e(v_i) = x(v_i)E \in \mathbb{R}^{1 \times d}.$$

Alkuperäinen esitysmatriisi muodostetaan näistä upotuksista riveittäin:

$$H_{i,:}^{[0]} = e(v_i) = x(v_i)E, \quad i = 1, \dots, T.$$

Tällöin $H^{[0]} \in \mathbb{R}^{T \times d}$. Laskenta vastaa luvussa 3 tarkasteltua tokenin muuntamista jatkuvaksi esitysvektoriksi.

Tässä tarkasteltavassa arkkitehtuurissa positiotieto sisällytetään itsehuomion kysely- ja avainvektoreihin RoPE-rotaatioiden avulla. Syötteen upotusvektoreihin ei siten lisätä erillisiä absoluuttisia positiovektoreita. Jatkossa dekodierilohkon kausaalisen monipäisen itsehuomion oletetaan sisältävän edellä määritellyt rotaatiot kunkin huomiopään kyselyille ja avaimille.

Olkoon dekodierilohkojen lukumäärä B . Esitysmatriisia muunnetaan peräkkäisissä lohkoissa seuraavasti:

$$H^{[b]} = \text{DecoderBlock}_b \left(H^{[b-1]} \right), \quad b = 1, \dots, B.$$

Kullakin lohkolle on omat opittavat parametrinsa, mutta esitysmatriisin koko säilyy kaikissa vaiheissa samana: $H^{[b]} \in \mathbb{R}^{T \times d}$. Hakasulkeinen yläindeksi ilmaisee tässä koko mallin dekodierilohkon indeksin, eikä yksittäisen lohkon sisäistä laskentavaihetta.

Kausaalisuus säilyy myös useita lohkoja yhdistettäessä. Ensimmäisessä lohkossa position i esitys voi hyödyntää ainoastaan positioita $1, \dots, i$. Seuraavassa lohkossa itsehuomio yhdistää vain näihin positioihin kuuluvia esityksiä, joista yksikään ei

sisällä tietoa positiota i seuraavista tokeneista. Koska normalisointi ja myötäkytkentäverkko toimivat positiokohtaisesti, ne eivät muuta tätä riippuvuusrakennetta. Siten position i esitys riippuu kaikissa lohkoissa vain kontekstista $v_{1:i}$.

Tarkasteltavassa pre-norm-rakenteessa viimeisen dekooderilohkon tulokseen sovelletaan vielä positiokohtaista normalisointia:

$$Z = \text{Norm}_{\text{out}}(H^{[B]}) \in \mathbb{R}^{T \times d}.$$

Olkoon $z_i = Z_{i,:} \in \mathbb{R}^{1 \times d}$ position i lopullinen esitys rivivektorina. Se muunnetaan sanaston ulottuvuuteen affiinilla kuvauksella

$$\ell_i = z_i W_{\text{out}} + b_{\text{out}},$$

missä $W_{\text{out}} \in \mathbb{R}^{d \times |\mathcal{V}|}$ ja $b_{\text{out}} \in \mathbb{R}^{1 \times |\mathcal{V}|}$ ovat opittavia parametreja. Joissakin toteutuksissa siirtovektori jätetään pois. Rivivektorin ℓ_i komponentteja kutsutaan logiteiksi. Ne ovat tokenien normalisoimattomia pistemääriä, joista muodostetaan todennäköisyysjakauma softmax-funktiolla.

Merkitään mallin kaikkia opittavia parametreja symbolilla θ . Seuraavan tokenin ehdollinen todennäköisyys on

$$p_{\theta}(v_{i+1} = v \mid v_{1:i}) = \frac{\exp(\ell_{i,v})}{\sum_{u \in \mathcal{V}} \exp(\ell_{i,u})}, \quad v \in \mathcal{V}.$$

Position i tulos ennustaa siis position $i + 1$ tokenia. Koska position i esitys ei riipu tulevista tokeneista, myös siitä muodostettu jakauma riippuu vain kontekstista $v_{1:i}$. Koko syötekontekstia seuraavan tokenin jakauma saadaan viimeisen position tuloksesta:

$$f_{\text{Tr}}(v_{1:T}) = \text{softmax}(\ell_T) \in \Delta(\mathcal{V}).$$

Kielimallin laskentaketju etenee näin diskreeteistä tokeneista jatkuviin esityksiin, kontekstuaalisiin muunnoksiin ja lopulta takaisin diskreetin sanaston yli määritellyksi jakaumaksi.

Arkkitehtuuri määrittää seuraavan tokenin jakauman annetuilla parametreilla θ . Seuraavaksi tarkastellaan, miten parametrit opitaan aineistosta ja miten opittua jakaumaa käytetään kokonaisten tokenijonojen muodostamiseen.

5.5 Autoregressiivinen koulutus ja generointi

Transformer-kielimallin koulutus perustuu samaan uskottavuusperiaatteeseen kuin aiemmin tarkasteltujen kielimallien koulutus. Ero on ehdollisten jakaumien parametrisoinnissa: Transformer muodostaa ne useiden itsehuomio- ja myötäkytkentäkerrosten avulla.

Todennäköisyyksien ketjusaännön perusteella tokenijonon $w_{1:T}$ todennäköisyys voidaan esittää muodossa

$$p_{\theta}(w_{1:T}) = \prod_{t=1}^T p_{\theta}(w_t \mid w_{<t}),$$

missä $w_{<t} = (w_1, \dots, w_{t-1})$.

Olkoon koulutusaineisto $\mathcal{D} = \{w^{(1)}, \dots, w^{(N)}\}$, missä sekvenssin $w^{(r)}$ ennustettavien tokenien lukumäärä on T_r . Tokenikohtainen keskimääräinen negatiivinen log-uskottavuus on

$$J(\theta) = -\frac{1}{\sum_{r=1}^N T_r} \sum_{r=1}^N \sum_{t=1}^{T_r} \log p_{\theta}(w_t^{(r)} \mid w_{<t}^{(r)}).$$

Kunkin position opetusvastaus on aineistossa havaittu seuraava token. Kun vastaus esitetään one-hot-jakaumana, yksittäisen position negatiivinen log-uskottavuus vastaa tämän jakauman ja mallin ennustejakauman ristientropiaa. Koulutuksessa minimoidaan siten samaa perustyyppistä tavoitefunktiota kuin luvussa 3, mutta sen gradientti kulkee nyt koko Transformer-verkon läpi.

Gradientit lasketaan vastavirta-algoritmillä ja parametreja päivitetään minierien perusteella gradienttipohjaisilla optimointimenetelmillä. Luvussa 4 käsitellyt menetelmät soveltuvat siten myös Transformer-kielimallin koulutukseen. Laajoilla tekstiaineistoilla toteutettava autoregressiivinen esikoulutus on keskeinen lähtökoh-ta dekooderipohjaisille kielimalleille [28].

Koulutuksessa kunkin ennusteen kontekstina käytetään aineistossa havaittua oikeaa prefiksiä $w_{<t}$ eikä mallin itsensä aiemmin tuottamia tokeneita. Tätä käytäntöä kutsutaan nimellä *teacher forcing*. Kunkin position opetusvastaus on aineistossa havaittu seuraava token.

Kaikkien positioiden ennusteet voidaan laskea yhdellä eteenpäinlaskennalla, koska oikeat syödetokenit tunnetaan koulutuksessa etukäteen. Kausaalinen maski estää silti kutakin positiota hyödyntämästä omaa ennustekohdettaan tai sitä myöhempiä tokeneita. Sekvenssin positioiden rinnakkainen käsittely ei siten riko autoregressiivistä mallinnusta [43].

Kausaalinen huomiomaski on erotettava häviön laskennassa käytettävästä maskista. Huomiomaski määrää, minkä positioiden välillä informaatio voi kulkea. Häviömaski puolestaan määrää, mitkä ennustepositiot sisällytetään tavoitefunktioon. Esimerkiksi eripituisten sekvenssien tasaamiseen käytettävät täytetokenit voidaan jättää häviön ulkopuolelle.

Esikoulutettua mallia voidaan edelleen mukauttaa ohjeiden noudattamiseen valvotulla ohjevirityksellä. Tällöin koulutusesimerkki koostuu syötekontekstista c ja tavoitevastauksesta $y = (w_1, \dots, w_K)$. Syötekonteksti voi sisältää esimerkiksi tehtävän ohjeen, käyttäjän kysymyksen ja muuta tehtävän ratkaisemiseen tarvittavaa tietoa. Kun häviö lasketaan vain vastaustokeneista, yhden esimerkin tavoite on

$$J_{\text{instr}}(\theta; c, y) = -\sum_{t=1}^K \log p_{\theta}(w_t \mid c, w_{<t}).$$

Syötekonteksti vaikuttaa vastaustokenien ennusteisiin, vaikka sen omista positioista ei tässä lasketa ennustehäviötä.

Ohjeviritys ja käyttöhetkellä annettu ohje ovat eri asioita. Ohjeviritys muuttaa mallin parametreja optimoinnin avulla. Pelkkä ohjeen, skeemakuvausten tai esimerkin lisääminen käyttöhetken kontekstiin ei muuta parametreja, vaan muuttaa sitä informaatiota, jonka perusteella malli muodostaa ehdollisen jakauman.

Generoinnissa oikeita tulevia tokeneita ei tunneta. Malli muodostaa ensin seuraavan tokenin jakauman nykyisen kontekstin perusteella. Jakaumasta valitaan token, joka liitetään kontekstin loppuun, minkä jälkeen laskenta toistetaan. Tuotettu token toimii siten seuraavien ennusteiden syötteenä.

Yksinkertainen valintasääntö on ahne valinta:

$$\hat{w}_{t+1} = \arg \max_{v \in \mathcal{V}} p_{\theta}(v \mid w_{1:t}).$$

Se valitsee jokaisessa vaiheessa kyseisen jakauman todennäköisimmän tokenin. Vaihtoehtona on otanta, jossa seuraava token arvotaan jakaumasta.

Otannassa jakauman keskittyneisyyttä voidaan muuttaa lämpötilaparametrilla $\tau > 0$:

$$p_{\theta,\tau}(v \mid w_{1:t}) = \frac{\exp(\ell_{t,v}/\tau)}{\sum_{u \in \mathcal{V}} \exp(\ell_{t,u}/\tau)}.$$

Kun $\tau = 1$, saadaan alkuperäinen softmax-jakauma. Pienempi lämpötila keskittää massaa suurimpien logittien kohdalle, kun taas suurempi lämpötila tasoittaa jakaumaa. Jos suurin logitti on yksikäsitteinen, jakauma keskittyy sen tokeniin lämpötilan lähestyessä nollaa.

Generointi päättyy esimerkiksi lopetustokeniin tai tuotokselle asetettuun pituusrajaan. Myös syötteen ja tuotettujen tokenien yhteinen kontekstinpituus rajoittaa laskentaa. Rajan saavuttamisen jälkeen jatkaminen edellyttää kontekstin lyhentämistä tai muuta erikseen määriteltä kontekstinhallintaa.

Ahne valinta maksimoi todennäköisyyden vain paikallisesti, eikä se yleensä tuota todennäköisintä kokonaista tokenijonoa. Vielä olennaisempaa on, ettei edes todennäköisin tokenijono välttämättä ole tehtävän kannalta oikea vastaus. Koulutustavoite palkitsee aineistossa havaittujen tokenien ennustamista, mutta ei sellaisenaan takaa tuotoksen loogista tai toiminnallista oikeellisuutta.

Tämä ero korostuu rakenteisten kyselykielten generoinnissa. SQL-kysely voi näyttää uskottavalta ja olla suoritettava, vaikka se vastaisi eri kysymykseen kuin käyttäjä tarkoitti.

5.6 Kielimalli Text-to-SQL-tehtävässä

Text-to-SQL-tehtävässä luonnollisella kielellä esitetty tiedontarve muunnetaan tietokantaan kohdistuvaksi SQL-kyselyksi. Autoregressiivisen kielimallin näkökulmasta kyseessä on ehdollinen generointitehtävä: tuotettavan tokenijonon tulee muodostaa SQL-kysely, jonka merkitys vastaa käyttäjän kysymystä käytettävissä olevan tietokannan ja sitä koskevien määritelmien puitteissa.

Olkoon q luonnollisen kielen kysymys, s tietokannan skeeman kuvaus ja m semanttinen lisätieto. Skeema kuvaa esimerkiksi taulut, sarakkeet ja niiden väliset suhteet. Semanttinen lisätieto täydentää tätä kuvausta käsitteiden merkityksillä, mittarien määritelmillä ja muilla tulkintasäännöillä.

Nämä tiedot esitetään mallille tokenikontekstina

$$c = \text{Serialize}(q, s, m),$$

missä Serialize tarkoittaa tietojen muuntamista mallin käsittelemäksi tokenijonoksi. Konteksti voi sisältää esimerkiksi tehtäväohjeen, skeeman tekstimuotoisen kuvauksen ja esimerkkejä kysymysten ja SQL-kyselyiden vastaavuuksista.

Kun SQL-kyselyn tokenijono on $y = (y_1, \dots, y_K)$, sen ehdollinen todennäköisyys voidaan kirjoittaa muodossa

$$p_{\theta}(y \mid c) = \prod_{t=1}^K p_{\theta}(y_t \mid c, w_{<t}).$$

Kontekstin ja generoinnissa tarvittavan historian on mahdollista mallin käytettävissä olevaan konteksti-ikkunaan. Laajan tietokannan tapauksessa koko skeeman ja kaiken lisätiedon sisällyttäminen yhteen syötteeseen ei siten välttämättä ole tarkoituksenmukaista.

Tämä esitys kuvaa kielimallin generointivaihetta, ei välttämättä koko Text-to-SQL-järjestelmän toimintaa. Järjestelmä voi lisäksi valita relevantteja skeeman osia, hakea esimerkkejä, tarkistaa tuotettua SQL-kyselyä tai muodostaa korjatun ehdotuksen uuden mallikutsun avulla. Sovelluksen toiminta on tällöin useiden käsittelyvaiheiden yhteistulos.

Keskeinen osatehtävä on skeemaan kohdistaminen (engl. *schema linking*), jossa kysymyksen ilmaukset yhdistetään tietokannan tauluihin, sarakkeisiin ja muihin rakenteisiin. Kysymyksessä käytetty sanasto ei välttämättä vastaa suoraan tietokannan nimiä. Lisäksi oikea vastaus voi edellyttää useiden taulujen yhdistämistä, vaikka käyttäjä ei mainitsisi niitä erikseen.

Tarkastellaan esimerkiksi kysymystä:

Mikä oli viime vuoden myynti asiakasryhmittäin?

Sen muuntaminen SQL-kyselyksi edellyttää vähintään myyntitapahtumia kuvaavan taulun, myyntimäärän sisältävän sarakkeen, ajallisen rajauksen ja asiakasryhmän tunnistamista. Jos asiakasryhmä sijaitsee eri taulussa kuin myyntitapahtumat, on lisäksi valittava oikea liitosavain ja liitospolku. Ryhmittely ja aggregointi on muodostettava siten, että tuloksen tarkkuustaso vastaa kysymystä.

Pelkkä skeeman tunteminen ei aina ratkaise näitä valintoja. Esimerkin sana *myynti* voi tarkoittaa bruttomyyntiä, nettomyyntiä tai laskutettua myyntiä. Samoin on ratkaistava, vähennetäänkö palautukset ja perustuuko ajallinen rajausta tilauspäivään vai laskutuspäivään. Ilmaus *viime vuosi* edellyttää viiteajankohtaa sekä tietoa siitä, tarkoitetaanko kalenterivuotta vai tilikautta.

Skeema ja semanttinen tieto täydentävät siten toisiaan. Skeema määrittää käytettävissä olevat rakenteet, kun taas semanttinen tieto auttaa tulkitsemaan, miten näitä rakenteita tulee käyttää tietyn tiedontarpeen ratkaisemiseen. Semanttiseksi kerrokseksi voidaan kutsua tietokannan rakenteita täydentävää kuvausta, jossa määritellään esimerkiksi liiketoimintakäsitteitä, mittareita ja hyväksytyjä taulujen välisiä suhteita.

Kielimallille annettu semanttinen kuvaus muuttaa generoinnin ehdollistavaa kontekstia. Se voi vähentää monitulkintaisuutta tarjoamalla eksplisiittisiä määritelmiä sen sijaan, että mallin olisi pääteltävä merkitykset pelkästään sarakkeiden nimistä tai esikoulutuksessa opituista yhteyksistä. Kuvaus ei kuitenkaan takaa oikeaa tulosta: malli voi soveltaa sitä väärin, ja virheellinen tai ristiriitainen kuvaus voi itsessään johtaa virheelliseen kyselyyn.

SQL-kyselyn oikeellisuudessa on hyödyllistä erottaa kolme tasoa. Ensimmäinen on syntaktinen kelvollisuus: noudattaako tuotettu kysely käytetyn SQL-murteen kieliooppia? Toinen on suoritettavuus: voidaanko kysely suorittaa kyseisessä tietokantaympäristössä? Kolmas on semanttinen oikeellisuus: toteuttaako kysely käyttäjän tarkoittaman tiedonhaun?

Syntaktisesti kelvollinen kysely voi viitata olemattomaan sarakkeeseen ja olla siksi suorituskelvoton. Suoritettava kysely voi puolestaan käyttää väärää aikarajausta tai liitosta ja tuottaa siten väärän vastauksen. Lisäksi suoritettavuuteen vaikuttavat kyselyn ulkopuoliset tekijät, kuten käyttöoikeudet ja ympäristön tila. Suoritusvirhe ei siten aina merkitse generoidun SQL-kyselyn loogista virhettä.

Olkon D tietokannan tila eli tietokantainstanssi. Merkitään onnistuneesti suoritettun kyselyn tulosta symbolilla

$$\text{Exec}(y, D).$$

Jos y^* on kysymykseen laadittu vertailukysely, tuotettua kyselyä voidaan arvioida vertaamalla suoritustuloksia:

$$\text{Exec}(y, D) = \text{Exec}(y^*, D).$$

Vertailussa käytettävä tulosten yhtäsuuruus on määriteltävä tehtävän mukaisesti. Esimerkiksi rivijärjestyksen merkitys riippuu siitä, edellyttääkö kysymys järjestettyä tulosta.

Pelkkä SQL-kyselyiden merkkijonojen yhtäsuuruus on liian rajoittava oikeellisuuden peruste, sillä eri tavoin kirjoitetut kyselyt voivat toteuttaa saman tiedonhaun. Toisaalta myöskään sama suoritustulos yhdellä tietokantainstanssilla ei yleisesti todista kyselyiden semanttista ekvivalenssia. Virheellinen suodatusehto voi esimerkiksi jäädä havaitsematta, jos aineistossa ei ole rivejä, joihin se vaikuttaisi.

Luonnollisen kielen kysymys ei myöskään aina määritä yksikäsitteistä tavoitekyselyä. Jos keskeinen mittari tai aikaväli on määrittelemätön, useat SQL-kyselyt voivat vastata kysymyksen eri tulkintoja. Tällöin tarkentavan kysymyksen esittäminen voi olla asianmukaisempaa kuin yhden tulkinnan valitseminen ilman perustelua. Jos tarvittavaa tietoa ei ole tietokannassa, oikea toiminta voi puolestaan olla perusteltu ilmoitus siitä, ettei kysymykseen voida vastata käytettävissä olevilla tiedoilla.

Autoregressiivinen kielimalli tarjoaa näin matemaattisen mekanismin SQL-kyselyiden muodostamiseen, mutta sen tokeniennusteisiin perustuva koulutustavoite ei yksin takaa kyselyiden oikeellisuutta. Käytännön onnistuminen riippuu myös skeeman esitystavasta, semanttisista määritelmistä, kysymyksen yksikäsitteisyydestä ja järjestelmän muista käsittelyvaiheista.

Seuraavassa luvussa määritellään tutkimusasetelma, jossa tätä kokonaisuutta tarkastellaan Cortex Analystiin perustuvassa Text-to-SQL-tapaustutkimuksessa. Edellä esitetty teoria toimii tehtävän käsitteellisenä perustana, mutta sitä ei tulkita kuvaukseksi palvelun kaikista sisäisistä toteutusratkaisuista.

6 Metodologia

6.1 Tutkimusasetelma

Tutkimus toteutetaan empiirisenä tapaustutkimuksena, jossa arvioidaan yhden kielimallipohjaisen järjestelmän toimintaa rajatussa sovelluskontekstissa. Tavoitteena ei ole rakentaa uutta mallia tai vertailla useita eri järjestelmiä keskenään, vaan tarkastella, miten Cortex Analyst suoriutuu käytännöllisestä tehtävästä, jossa luonnollisen kielen kysymykset muunnetaan SQL-kyselyiksi. Tapaustutkimus soveltuu tähän tarkoitukseen hyvin, koska se mahdollistaa järjestelmän toiminnan yksityiskohtaisen tarkastelun todellisessa kaupallisessa ympäristössä.

Tutkimuksen analyysiyksikkönä on yksittäinen käyttäjän luonnollisen kielen kysymys ja sitä vastaava Cortex Analystin tuottama SQL-kysely. Arviointi kohdistuu siihen, vastaako muodostettu kysely käyttäjän tarkoitusta, hyödyntääkö se oikeita tauluja ja kenttiä sekä noudattaako se tietokannan rakennetta. Näin tutkimus keskittyy suoraan siihen vaiheeseen, jossa kielimallin tulkinta muuttuu koneellisesti suoritettavaksi tietokantakyselyksi.

6.2 Tutkimusaineisto

Tutkimuksen aineistona toimii puhelinkeskusdataa sisältävä tietokanta, joka noudattaa kuvassa 1 esitettyä rakennetta. Tarkastelun keskiössä on yksi keskusteludataa sisältävä päätason taulu, jota täydentävät kolme liitännäistaulua. Liitännäistaulujen avulla voidaan tarkentaa päätason taulun yksittäisten kenttien sisältöä ja merkitystä. Tietokannan rakenne on siten riittävän monipuolinen, jotta voidaan tarkastella sekä yksinkertaisia että vaativampia kyselytilanteita, kuten taululiitoksia, suodatuksia ja aggregaatioita.

Aineisto sisältää kaupallisesti arkaluonteista tietoa, minkä vuoksi varsinaisia kyselytuloksia ei julkaista. Tästä syystä tutkimuksessa vertaillaan järjestelmän tuottamia SQL-kyselyitä eikä niiden palauttamia tietosisältöjä. Ratkaisu tukee myös analyysin johdonmukaisuutta, koska virheellisen kyselyn vaikutus lopputulokseen ei ole aina pääteltävissä pelkästään syntaksista, vaan riippuu osittain tietokannan sisällöstä.

6.3 Cortex Analyst ja semanttinen näkymä

Tutkimuksessa käytettävä järjestelmä on Snowflaken Cortex Analyst [39]. Järjestelmän tarkoituksena on mahdollistaa tietokantatiedon hyödyntäminen luonnollisen kielen kautta siten, että käyttäjän ei tarvitse tuntea SQL-kieltä tai tietokannan skeemaa yksityiskohtaisesti. Cortex Analyst muodostaa käyttäjän kysymyksestä SQL-kyselyn ja voi suorittaa sen tietokantaa vasten.

Cortex Analystin toiminta perustuu semanttiseen näkymään, jossa tietokannan taulut, kentät ja niiden merkitykset kuvataan YAML-muotoisena määrittelynä (ks. kuva 2). Semanttinen näkymä tarjoaa kielimallille rakenteellisen kuvauksen siitä, mitä tietoa tietokannassa on saatavilla ja miten eri taulut liittyvät toisiinsa. Näin järjestelmä pystyy muodostamaan syntaktisesti valideja SQL-kyselyitä ja kohdistamaan kyselyn oikeisiin tietokannan osiin. Semanttisen näkymän laatu on keskeinen

osa järjestelmän toimivuutta, koska kielimallin tulkinta perustuu olennaisesti siihen, miten skeema on kuvattu.

Tutkimuksessa käytetty semanttinen näkymä on tutkijan rakentama yhteistyössä yksityisen IT-konsulttiyrityksen Knowitin ja yhden heidän asiakkaan kanssa osana hanketta, jossa pyritään hyödyntämään kielimalleja helpottamaan tietokannan käyttöä.

Tässä tutkimuksessa käytetty semanttinen näkymä kuvailee kaikki kuvassa 1 esiintyvät taulut, niiden kentät ja suhteet toisiinsa sekä useita kentistä laskettuja metriikoita. Kuvassa 2 on esitetty yksinkertaistettu versio käytetystä semanttisesta näkymästä.

```
name: SEM_EXAMPLE
description: |-
  Semantic model that models conversations data from Genesys
  Cloud contact center software.
tables:
  - name: CONV
    description: |-
      All Genesys conversations.
    base_table:
      database: DEV
      schema: DBT_OM
      table: FACT_GENESYS__CONVERSATIONS_STANDARD
    dimensions:
      - name: C_ADDRESS_FROM
        description: Source address for the interaction. Only
          present in email contacts..
        expr: conv.address_from
        data_type: VARCHAR(16777216)
    facts:
      - name: CALL_DURATION
        description: Duration of the actual call in seconds.
          The time the customer speaks to an agent
        expr: conv.call_duration
        data_type: NUMBER(38,0)
        access_modifier: public_access
    metrics:
      - name: AVERAGE_CALL_DURATION
        description: Average call duration in seconds
        expr: sum(conv.call_duration)
        access_modifier: public_access
    primary_key:
      columns:
        - PARTICIPANTS_ID
        - IDENTIFIER
```

Kuva 2: Esimerkki semanttisesta näkymästä. Esimerkki on yksinkertaistettu versio tutkimuksessa käytetystä näkymästä.

Kokeiden aikana Cortex Analyst käytti taustallaan Anthropicin Claude Sonnet 4 -mallia [1]. Tutkimuksen tulokset kuvaavat ensisijaisesti juuri tämän järjestelmän ja mallin yhdistelmän toimintaa. Huhtikuussa 2026 Cortex Analyst voi käyttää uudempaa Claude Sonnet 4.6 -mallia [39]. Tuloksia ei siksi voida sellaisenaan yleistää kaikkiin kielimalleihin tai muihin luonnollisen kielen tietokantarajapintoihin.

6.4 Arviointiperusteet

Tuotettuja SQL-kyselyitä arvioidaan useiden toisiaan täydentävien kriteerien avulla. Ensinnäkin tarkastellaan, valitseeko järjestelmä oikeat taulut ja käyttääkö se oikeita liitoksia silloin, kun käyttäjän kysymys edellyttää useamman taulun yhdistämistä. Toiseksi arvioidaan suodatusehtoja, erityisesti where-lauseiden sisältöä, koska pienikin virhe niissä voi muuttaa kyselyn merkitystä olennaisesti. Lisäksi huomioidaan aggregaatiot, duplikaattien käsittely ja sarakevalinnat.

Syntaktista validiutta ei tarkastella erikseen, koska kaikki järjestelmän tuottamat kyselyt olivat syntaktisesti valideja.

Arvioinnissa erotetaan toisistaan neljä poikkeamatyyppiä. Kysely luokitellaan korrektiksi, jos sen perusteella saatava vastaus vastaisi esitettyyn luonnollisen kielen kysymykseen oikein ja huomioisi ne implisiittiset taustaoletukset, jotka ovat sovellusalueen näkökulmasta perusteltuja.

Tutkittava tietokanta sisältää puhelinkeskusdataa, joten erityisesti puhelun suuntaan liittyvät taustaoletukset ovat keskeisiä. Esimerkiksi palvelutasoa koskevan kysymyksen implisiittisenä taustaoletuksena pidetään sitä, että tarkastelu kohdistuu vain saapuviin kontakteihin. Korrekti kysely huomioi tällaiset taustaoletukset ja vastaa käyttäjän tiedontarpeeseen oikein.

Kysely merkitään melkein korrektiksi, jos generoitu SQL vastaa olennaisilta osin kysymykseen, mutta sisältää teknisen puutteen. Esimerkiksi kysely, josta puuttuu NULLIF -lause jakolaskun nimittäjästä, vastaa sisällöllisesti samaan kysymykseen kuin NULLIF-avainsanan sisältävä kysely, mutta voi erikoistapauksessa johtaa virheeseen. Tällainen kysely luokitellaan melkein korrektiksi.

Kysely luokitellaan virheelliseksi, jos se ei vastaa kysymykseen merkityksellisellä tavalla. Esimerkiksi kysely, joka käyttää väärää suodatusta tai josta puuttuu olennainen taulu tai kenttä, merkitään virheelliseksi.

Jos malli ei tuota lainkaan SQL-kyselyä, vaan ilmoittaa, että kysymykseen ei voida vastata, tämä luokitellaan erikseen ei kyselyä -kategoriaksi. Kaikkiin tutkimuksessa käytettyihin kysymyksiin on mahdollista vastata käytettävissä olevan aineiston ja semanttisen näkymän perusteella, joten tähän kategoriaan kuuluvat vastaukset ovat virheellisiä. Ne on kuitenkin mielekästä erottaa muista virheellisistä vastauksista, koska ne eivät tuota harhaanjohtavia tuloksia.

Samaan luonnollisen kielen kysymykseen voi olla useita oikeellisia SQL-vastauksia. Esimerkiksi kysymykseen How many contacts did we handle in January 2026? järjestelmä voi tuottaa kyselyn, joka palauttaa yhden kokonaisluvun, tai kyselyn, joka palauttaa taulun sisältäen kontaktien määrän tammi-, helmi- ja maaliskuulle, joista käyttäjä voi poimia vastauksen. Tässä tutkielmassa kysely tulkitaan oikeelliseksi siinä tapauksessa, että sen tuottama vastaus sisältää käyttäjän tiedontarpeen.

Tästä johtuen oikeellisuuden lisäksi tutkimuksen keskeinen kiinnostuksen kohde on kielimallin tuottamien kyselyiden vaihtelu. Tämän vuoksi analysoidaan myös, kuinka monta uniikkia SQL-kyselyä järjestelmä tuottaa kutakin kysymystä kohden, kun sama luonnollisen kielen kysymys tai jokin sen variaatioista esitetään useita kertoja.

Kyselyä ei pidetä uniikkina, jos semanttisesti ekvivalentti kysely on tuotettu aiemmin, paitsi siinä tapauksessa, että toinen kysely hyödyntää semanttisessa näkymässä valmiiksi määriteltä metriikkaa ja toinen tuottaa laskukaavan, joka on ekvivalentti semanttisessa näkymässä määritellyn metriikan kanssa. Esimerkiksi jos semanttiseen näkymään ollaan määritelly metriikka `total_calls` kaavalla `select sum(*) from conv`, niin kyselyt `select sum(*) from conv` ja `select total_calls from semantic_view` nähdään toisistaan uniikkeina.

Päätös perustellaan sillä, että nämä kaksi lähestymistapaa ovat olennaisesti erilaiset (käytettävissä olevan kontekstin soveltaminen vs. päättely). Vaikka kyselyt ovat sisällöllisesti semanttisesti samankaltaisia, niiden muodostaminen edellyttää erilaista ongelmanratkaisua ja hyödyntää järjestelmältä eri kyvykkyyksiä. Ensimmäisessä tapauksessa malli tunnistaa valmiiksi määritellyn metriikan ja osaa käyttää sitä suoraan, kun taas jälkimmäisessä tapauksessa sen on pääteltävä tarvittava laskentatapa. Tämän vuoksi tapaukset erotellaan analyysissa toisistaan, vaikka niiden semanttinen tavoite on sama.

Tuotettujen SQL-kyselyiden oikeellisuus arvioitiin manuaalisesti vertaamalla niitä tutkijan tulkintaan siitä, millainen kysely vastaisi käyttäjän tiedontarvetta. Arvioinnin suoritti yksi arvioija, mikä heikentää luokittelun reliabiliteettia.

7 Kokeet

Ensimmäisessä kokeessa tarkastellaan Cortex Analystin konsistenssia, eli sitä, tuottaako järjestelmä saman SQL-kyselyn, kun sama luonnollisen kielen kysymys esitetään useita kertoja.

Toisessa kokeessa tarkastellaan järjestelmän herkkyyttä sellaisille syötteen muutoksille, joiden ei pitäisi muuttaa kysymyksen merkitystä. Näissä kokeissa muodostetaan useita eri sanamuotoja samasta tiedontarpeesta ja arvioidaan, tuottaako järjestelmä niistä olennaisesti saman SQL-kyselyn.

Konsistenssikoe Koska kielimallit ovat tilastollisia järjestelmiä, sama syöte ei välttämättä aina johda täsmälleen samaan tulosteeseen. Tietokantakyselyiden tapauksessa tämä on olennainen kysymys, koska käyttäjät odottavat järjestelmältä ennustettavaa ja toistettavaa toimintaa.

Konsistenssikokeessa sama luonnollisen kielen kysymys esitetään järjestelmälle 10 kertaa. Jokaisesta suorituksesta tallennetaan järjestelmän tuottama SQL-kysely, joita verrataan keskenään. Käytetyt kysymykset ovat seuraavat:

- A. *How many total contacts did we handle in January 2026?*
- B. *What's our answer rate for calls in February 2026?*
- C. *What percentage of contacts that were answered within our service level target were closed by the customer versus closed by the agent?*
- D. *What was our peak contact hours by day of week? Show me the distribution of when contacts arrive throughout the day, broken down by hour and weekday.*
- E. *What's the success rate of our callbacks? Show me how many callbacks resulted in a conversation versus no answer, and what the average time between the original inbound contact and the callback was.*

Määrittelyssä semanttisessa näkymässä on kuvattu metriikoihin SQL-kyselyt, jotka vastaavat suoraan kysymyksiin A ja B. Kysymykset C-E ovat huomattavasti monimutkaisempia, ja niihin ei ole ennalta validoituja SQL-kyselyitä semanttisessa näkymässä. Kysymys D vaatii, että kentästä `CONTACT_CONNECTED_DATETIME` erotetaan kellonaika tunnin tarkkuudella, ja viikonpäivä pitää hakea kalenteritaulusta. Kysymys E on rakenteeltaan monitulkintainen. Erityisesti takaisinsoiton ja alkuperäisen sisääntulevan kontaktin välisen ajan keskiarvoa koskeva osa voidaan tulkita vaativan kahden eri rivin välistä vertailua, vaikka käytetyssä aineistossa vastaus voidaan muodostaa yhdeltä takaisinsoittoa kuvaavalta riviltä.

Sensitiivisyyskoe Kokeen tavoitteena on arvioida, tulkitseeko Cortex Analyst semanttisesti saman sisältöiset kysymykset johdonmukaisesti vai johtavatko kielelliset erot erilaisiin SQL-kyselyihin. Jos järjestelmä on hyvin herkkä muotoilun vaihtelulle, luonnollisen kielen käyttö rajapintana muuttuu käyttäjän kannalta epävarmaksi. Jos taas järjestelmä tuottaa olennaisesti saman kyselyn eri sanamuodoista huolimatta, tämä tukee sen käytettävyyttä käytännön työssä.

Koe toteutetaan muodostamalla kustakin konsistenssikokeessa käytetystä kysymyksestä 4 vaihtoehtoisia versiota, joissa pyritään säilyttämään kysymyksen merkitys, mutta muokkaamaan sanamuotoa. Järjestelmän tuottamat SQL-kyselyt tallennetaan ja vertaillaan keskenään. Käytetyt kysymykset ovat seuraavat:

A. Variaatioita kysymyksestä A:

1. *What was the total number of contacts we handled in January 2026?*
2. *How many contacts did we handle overall in January 2026?*
3. *What is the total contact volume for January 2026?*
4. *How many total customer contacts were handled during January 2026?*

B. Variaatioita kysymyksestä B:

1. *What was our call answer rate in February 2026?*
2. *What percentage of incoming calls were answered in February 2026?*
3. *What percentage of calls were answered in February 2026?*
4. *How did our call answer rate perform during February 2026?*

C. Variaatioita kysymyksestä C:

1. *Of the contacts answered within our service-level target, what share were closed by the customer compared to those closed by the agent?*
2. *For interactions resolved within the service level target, what percentage were customer-closed versus agent-closed?*
3. *Within our service level target, how is the closure split: what percent of contacts were closed by customers versus agents?*
4. *Among contacts answered within SLA, what proportion ended with customer closure versus agent closure?*

D. Variaatioita kysymyksestä D:

1. *When do contacts most frequently arrive during the week? Break this down by weekday and hour.*
2. *Show the hourly contact-arrival distribution for every day of the week, highlighting which hours were highest on each weekday.*
3. *What times during the day (hour-by-hour) drive the most contacts, split by weekday. Please summarize peak hours and the overall distribution.*
4. *Across the week, when do contacts arrive most often? Provide an hour-by-hour breakdown for each weekday and identify the peak contact hours.*

E. Variaatioita kysymyksestä E:

1. *What percentage of our callback attempts lead to an actual conversation? Please break down callbacks that reached a conversation vs. those with no answer, and calculate the average delay from the original inbound to the callback.*

2. *How effective are our callbacks? Show the success rate by counting callbacks that resulted in a conversation versus no answer, and report the average time-to-callback from the initial inbound contact.*
3. *For our callback campaign, what's the outcomes split and average response speed? Provide counts of conversations vs. no-answer callbacks, the resulting success rate, and the average elapsed time between inbound contact and callback.*
4. *Can you analyze callback performance for me: success rate (conversation vs no answer) with the number of callbacks in each outcome category, plus the average time lag from the inbound contact to when the callback occurred?*

Jokainen kysymys molemmissa kokeissa esitetään järjestelmälle 10 kertaa ja jokainen yksittäinen suoritus tehdään tyhjällä sessiolla. Testit suoritettiin 23.03.2026-28.03.2026 välisenä aikana käyttäen samaa semanttista mallia.

Kymmenen toistoa valittiin käytännöllisenä kompromissina, joka mahdollistaa vaihtelun havaitsemisen ilman kohtuuttoman suurta manuaalisen arvioinnin työmäärää.

8 Tulokset

Merkitään konsistenssikokeen kysymykset A-E ja herkkyysskokeen variaatiot A1-A4, B1-B4, C1-C4, D1-D4 ja E1-E4 listojen 7 ja 7 mukaisesti. Kokeiden tulokset on esitetty taulukoissa 2 ja 3. Taulukot kuvaavat kunkin kysymyksen osalta, kuinka monta kertaa järjestelmän tuottama SQL-kysely oli korrekti, melkein korrekti, virheellinen tai johti tilanteeseen, jossa SQL-kyselyä ei tuotettu lainkaan.

Konsistenssikokeessa vain kysymys C tuotti sekä korrekteja että virheellisiä SQL-kyselyitä. Kysymys B tuotti kaksi uniikkia SQL-kyselyä, joista toinen käytti semanttisessa näkymässä määriteltäviä metriikkaa `answer_percentage` suoraan ja toinen laski vastaavan prosenttiluvun erillisellä laskukaavalla.

Kysymykset A, D ja E tuottivat konsistenssikokeessa vain yhden uniikin lopputuloksen kymmenessä suorituksessa. Vaikka kysymys E tuotti jokaisella suorituskerralla virheellisen vastauksen siinä mielessä, että SQL-kyselyä ei muodostettu, konsistenssin näkökulmasta järjestelmä oli johdonmukainen, koska tuloste oli joka kerta sama. Kysymys E osoittaa, että konsistenssi ei yksin riitä kuvaamaan järjestelmän luotettavuutta, mikä heijastaa Text-to-SQL-evaluoinnissa tunnettua eroa toistettavuuden (konsistenssin) ja semanttisen virheettömyyden välillä [27].

Tulokset viittaavat siihen, että tässä tutkimusasetelmassa havaitut ongelmat olivat usein toistettavia, mikä on hyödyllinen havainto sekä käyttäjän että semanttisen näkymän kehittäjän näkökulmasta. Kysymys C kuitenkin osoittaa, että järjestelmä pystyi tuottamaan samassa asetelmassa sekä korrekteja että virheellisiä vastauksia, joten toistettavuus ei ollut täydellistä.

Alla on esimerkki kysymykseen E tuotetusta ei kyselyä -vastauksesta:

I'm sorry, but this question cannot be fully answered with the available data. While we can determine how many callbacks were answered versus not answered (using the `c_answered` field filtered for callback media type), the semantic model does not contain information about the original inbound contact that triggered a callback. There is no field linking a callback conversation back to its originating inbound contact, so we cannot calculate the time between the original inbound contact and the callback attempt.

Kysymykseen B järjestelmä tuotti kaksi uniikkia SQL-kyselyä. Näistä toinen käytti semanttisessa näkymässä ennalta määriteltäviä metriikkaa `answer_percentage` ja toinen muodosti vastaavan laskukaavan eksplisiittisesti SQL:ssä. Kysymykseen C järjestelmä tuotti kaksi uniikkia SQL-kyselyä, joista toinen oli korrekti ja toinen virheellinen. Korrekti kysely tuotettiin seitsemän kertaa ja virheellinen kolme kertaa. Virheellisessä kyselyssä järjestelmä sekoitti asiakkaan ja agentin roolit keskenään.

Konsistenssikokeen 50 suorituksesta 37 tuotti korrektin SQL-kyselyn, 10 johti tilanteeseen, jossa järjestelmä ei tuottanut lainkaan SQL-kyselyä, ja 3 tuotti virheellisen SQL-kyselyn. Tulokset osoittavat, että Cortex Analyst pystyy usein tuottamaan johdonmukaisia SQL-kyselyitä, mutta sen luotettavuus ei ollut kaikissa kysymystyypeissä riittävällä tasolla.

Herkkyystesteissä suurin variaatio havaittiin kysymyksen C eri muotoiluissa, joissa järjestelmä tuotti yhteensä kuusi uniikkia SQL-kyselyä. Kun Cortex Analystille

syötettiin kysymyksen C eri variaatioita, järjestelmä tuotti väärän vastauksen 14 kertaa 50 suorituksesta. Virheellisissä kyselyissä toistuvien ongelmien oli se, että järjestelmä ei rajannut hakua sarakkeen `direction` mukaisesti, vaan käytti saraketta `direction_original` tai jätti suuntasuodatuksen kokonaan pois.

Kysymyksessä C ja sen variaatioissa puhutaan palvelutasosta, jolloin implisiittisenä taustaoletuksena on, että tarkastelu kohdistuu vain saapuviin kontakteihin. Havainto viittaa siihen, että Cortex Analyst ei tässä tutkimusasetelmassa aina huomionnut tällaisia implisiittisiä oletuksia johdonmukaisesti. Luonnollisen kielen kysymyksissä esiintyvät alimääritellyt ilmaukset ja implisiittisen toimialatiedon päättely muodostavatkin tunnetun haasteen Text-to-SQL-malleille [11, 6].

Kun kysymys E ja sen variaatiot annettiin järjestelmälle syötteenä, järjestelmä tuotti 40 kertaa 50 suorituksesta tekstivastauksen, jossa se ilmoitti, että kysymykseen ei voida vastata. Variaatio E2 tuotti yhdeksän kertaa korrektein SQL-kyselyn ja kerran kyselyn, joka oli muuten oikea mutta käytti `distinct(identifier)`-ehdon sijaan ehtoa, jossa koko rivi vaadittiin uniikiksi. Tämä on huomattavasti heikompi ehto ja johti duplikaattivirheisiin.

Kysymyksen E variaatio E2 ei implikoi yhtä vahvasti, että kysymykseen vastaaminen vaatisi kahden rivin välistä vertailua. Tämä voi osaltaan selittää, miksi juuri tämä variaatio tuotti enemmän oikeita vastauksia kuin muut.

Kysymys E oli Cortex Analystille vaikein, mikä painottaa virhetyyppien jakaumaa ei kyselyä -luokan suuntaan.

Taulukosta 4 nähdään, että muista virhetyypeistä yleisimpiä olivat suodatusvirheet ja duplikaattien hallintaan liittyvät virheet. Kaikki suodatusvirheet liittyivät sarakkeisiin `direction` ja `direction_original`, mikä viittaa siihen, että järjestelmä voisi olla robustimpi, jos nämä kaksi saraketta erotettaisiin toisistaan semanttisesti selvemmin. Esimerkiksi `direction_original`-sarakkeen korvaaminen `is_callback`-sarakkeella saattaisi helpottaa mallia erottamaan sarakkeiden merkitykset toisistaan. Tätä ei kuitenkaan testattu tässä tutkimuksessa.

On kuitenkin huomioitava, että juuri nämä sarakkeet nousivat esiin tässä tutkimusasetelmassa. Toisenlaisessa tietokannassa tai toisessa sovellusympäristössä ongelmalliset kentät voisivat olla erilaisia. Havainto korostaa semanttisen näkymän merkitystä järjestelmän luotettavuuden kannalta. Semanttisen näkymän huolellinen suunnittelu, metadatan rikastaminen ja iteratiivinen kehittäminen voivat pitkällä aikavälillä parantaa järjestelmän toimintaa merkittävästi [38, 13].

Duplikaattien hallintavirheillä tarkoitetaan tässä virheitä, joissa `distinct`-avainsanaa on käytetty tarpeettomasti tai väärin. Tutkimusasetelmassa vain kysymystä A vastaavissa SQL-kyselyissä `distinct`-avainsanaa ei tarvita. Muiden kysymysten kohdalla duplikaattien hallinta tulisi yleensä kohdistaa `identifier`-sarakkeeseen. Duplikaattien hallintavirheitä esiintyi tutkimuksessa yhteensä 18 kertaa 250 suorituksen aikana.

Kysymyksessä A duplikaattien hallintaa ei odoteta lainkaan, mutta 10 suorituksessa järjestelmä tuotti SQL-kyselyn, joka sisälsi lausekkeen `distinct(identifier)`. Muut havaitut duplikaattien hallintavirheet liittyivät siihen, että `distinct`-avainsanaa sovellettiin n-tupleen, joka sisälsi `identifier`-sarakkeen lisäksi myös muita sarakkeita. Tällöin kysely vaatii näiden sarakkeiden yhdistelmän olevan uniikki, mikä on heikompi ehto kuin se, että `identifier`-sarake olisi jo yksin uniikki.

Taululiitos- tai sarakevalintavirheitä ei havaittu. Tämä viittaa siihen, että järjestelmä tunnisti kysymyksistä yleensä oikein, mitä attribuutteja käyttäjä halusi hakea, vaikka se ei aina onnistunut tuottamaan oikeaa SQL-kyselyä niiden perusteella.

Taulukko 2: Konsistenssikokeen tulokset.

Kysymys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Erilaisten SQL- kyselyiden määrä
A	10	0	0	0	1
B	10	0	0	0	2
C	7	0	3	0	2
D	10	0	0	0	1
E	0	0	0	10	1

Taulukko 3: Herkkyystestien 1–5 tulokset. Sarakkeet ilmoittavat oikeiden, pienen virheen sisältävien, väärin ja ei kyselyä -vastausten lukumäärät sekä yhteenlaskettu uniikkien SQL-kyselyiden määrä. (*Yht.*).

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
A	10	0	0	0	1
A1	10	0	0	0	1
A2	10	0	0	0	1
A3	10	0	0	0	1
A4	0	10	0	0	2

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
B	10	0	0	0	2
B1	10	0	0	0	2
B2	10	0	0	0	2
B3	10	0	0	0	2
B4	10	0	0	0	2

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
C	7	0	3	0	2
C1	8	2	0	0	3
C2	10	0	0	0	3
C3	0	0	10	0	5
C4	8	1	1	0	6

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
D	10	0	0	0	1
D1	10	0	0	0	1
D2	10	0	0	0	1
D3	10	0	0	0	2
D4	7	0	3	0	3

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
E	0	0	0	10	1
E1	0	0	0	10	1
E2	9	1	0	0	3
E3	0	0	0	10	3
E4	0	0	0	10	3

Taulukko 4: Esiintyneiden virhetyyppien määrät kaikissa kokeissa. Rivit kuvaavat, kuinka monta kertaa kukin virhetyyppi esiintyi kussakin kysymyksessä tai sen variaatioissa. Jos yhdessä SQL-kyselyssä esiintyy useampi virhetyyppi, kysely lasketaan jokaiseen virhetyyppiin erikseen.

Kysymys	A	B	C	D	E	Yhteensä
Aggregaatio	0	0	2	0	0	2
Duplikaattien hallinta	10	0	7	0	1	18
Ei kyselyä	0	0	0	0	40	40
Sarakevalinta	0	0	0	0	0	0
Suodatus	0	0	11	3	0	14
Taululiitos	0	0	0	0	0	0

9 Rajoitukset

Tutkimukseen liittyy useita rajoituksia, jotka on huomioitava tuloksia tulkittaessa.

Ensinnäkin tutkimus kohdistuu yhteen järjestelmään, Snowflaken Cortex Analyysiin, jonka taustalla toimii Anthropicin Claude Sonnet 4 -malli. Tulokset kuvaavat siten yhtä tiettyä toteutusta eivätkä ole suoraan yleistettävissä muihin kielimalleihin, SQL-avustajiin tai luonnollisen kielen analytiikkatyökaluihin.

Toiseksi tutkimus perustuu yhteen sovellusalueeseen eli puhelinkeskusdataan. Tämän tietokannan rakenne, sanasto ja analyysitarpeet voivat poiketa merkittävästi muiden toimialojen vastaavista ympäristöistä. On mahdollista, että järjestelmä suoriutuisi eri tavoin esimerkiksi talousdatan, lokidatan tai asiakastietojen yhteydessä.

Kolmanneksi tutkimuksessa tarkastellaan mallin tuottamia SQL-kyselyitä eikä kyselyiden palauttamia varsinaisia vastauksia. Tämä rajaus on perusteltu aineiston luottamuksellisuuden vuoksi, mutta samalla se tarkoittaa, että tutkimus ei mittaa suoraan liiketoiminnallisen lopputuloksen oikeellisuutta. Kysely voi olla rakenteeltaan puutteellinen tai vaihtoehtoisesti syntaktisesti erilainen mutta käytännössä toimiva, eikä tätä eroa voida aina arvioida täysin ilman varsinaista tulosaineistoa.

Neljänneksi semanttisen näkymän laatu vaikuttaa suoraan siihen, miten hyvin Cortex Analyst kykenee muodostamaan kyselyitä. Jos näkymässä on epäselvyyksiä, puutteita tai monitulkintaisia kuvauksia, osa havaituista ongelmista voi johtua itse määrittelystä eikä pelkästään kielimallin rajoituksista. Tämän vuoksi tutkimus arvioi samalla myös sitä, kuinka hyvin kyseinen semanttinen näkymä tukee luonnollisen kielen ja tietokantarakenteen välistä tulkintaa.

Viidenneksi kielimallien toimintaa luonnehtii tietty epädeterministisyys, minkä vuoksi tulokset voivat vaihdella eri suorituskerroilla tai järjestelmäversioiden välillä. Lisäksi kaupalliset kielimallipohjaiset järjestelmät voivat muuttua ajan myötä ilman, että kaikki muutokset ovat käyttäjälle näkyviä. Tämän vuoksi tutkimuksen havainnot kuvaavat järjestelmän toimintaa tietyssä ajankohtana ja tietyssä konfiguraatiossa eivätkä välttämättä pysyvää ominaisuutta [8].

Kuudenneksi tuotettujen SQL-kyselyiden oikeellisuuden arvioi yksi arvioija. Tämä heikentää luokittelun reliabiliteettia ja lisää mahdollisuutta siihen, että osa luokitteluista sisältää arvioijan tulkintaan liittyvää subjektiivisuutta.

10 Johtopäätökset

Tämän tutkielman tavoitteena oli arvioida, kuinka hyvin Snowflaken Cortex Analyst soveltuu luonnollisen kielen tietokantakyselyihin puhelinkeskusdataa sisältävässä tietokantaympäristössä. Tarkastelun kohteena olivat erityisesti järjestelmän kyky tuottaa oikeellisia SQL-kyselyitä, sen konsistenssi tilanteissa, joissa sama kysymys esitetään useita kertoja, sekä sen herkkyyssellaisille syötteen muotoilun muutoksille, joiden ei pitäisi muuttaa kysymyksen merkitystä.

Tulokset osoittavat, että Cortex Analyst kykenee monissa tapauksissa tuottamaan oikeellisia ja keskenään johdonmukaisia SQL-kyselyitä. Erityisesti yksinkertaisemmat kysymykset sekä sellaiset kysymykset, joihin oli määritelty valmiita metriikoita semanttisessa näkymässä, tuottivat hyvin vakaita tuloksia. Tämä viittaa siihen, että kielimallipohjainen luonnollisen kielen rajapinta voi toimia käyttökelpoisena välineenä tietokannan hyödyntämisessä silloin, kun kysymykset ovat rakenteeltaan selkeitä ja semanttinen näkymä tukee niitä riittävän hyvin.

Tutkimus osoitti kuitenkin myös merkittäviä rajoituksia. Järjestelmän suorituskky heikkeni erityisesti silloin, kun kysymykseen sisältyi implisiittisiä taustaoletuksia tai kun oikean SQL-kyselyn muodostaminen edellytti tarkkaa suodatusta, duplikaattien hallintaa tai monitulkintaisen sanamuodon tulkintaa. Nämä havainnot ovat linjassa aiemman Text-to-SQL-tutkimuksen kanssa, jossa vastaavien monimutkaisten ehtojen ja taustatietojen yhdistämisen on todettu olevan kielimalleille huomattava haaste [19, 49]. Lisäksi havaittiin, että semanttisesti lähes saman sisältöiset kysymykset saattoivat johtaa erilaisiin SQL-kyselyihin ja joissakin tapauksissa myös erilaisiin virheisiin. Tämä ilmentää kielimalleille tyypillistä syöteherkkyyttä (*prompt sensitivity*), jossa pienetkin sanamuodon vaihtelut voivat muuttaa tuotettua lopputulosta merkittävästi [7], mikä heikentää luonnollisen kielen rajapinnan ennustettavuutta käyttäjän näkökulmasta.

Keskeinen havainto on, että järjestelmän luotettavuus ei riipu yksinomaan taustalla olevasta kielimallista, vaan myös siitä, miten hyvin semanttinen näkymä on suunniteltu. Tutkimuksessa havaitut virheet liittyivät usein tilanteisiin, joissa tietokannan kenttien merkitysten erottelu ei ollut mallin näkökulmasta riittävän selkeä. Tämän perusteella voidaan päätellä, että semanttinen näkymä ei ole pelkästään tekninen apurakenne, vaan keskeinen osa koko järjestelmän toimintaa. Sen huolellinen suunnittelu, yksiselitteinen nimeäminen ja iteratiivinen kehittäminen voivat parantaa luonnollisen kielen kyselyjärjestelmän luotettavuutta merkittävästi.

Tutkimuksen perusteella konsistenssia ei myöskään voida pitää yksinään riittävänä laadun mittarina. Järjestelmä saattoi tuottaa saman tuloksen toistuvasti myös silloin, kun tulos oli virheellinen tai kun SQL-kyselyä ei tuotettu lainkaan. Toisaalta joissakin tapauksissa samaan kysymykseen saatiin useita erilaisia SQL-kyselyitä, joista osa oli oikeellisia ja osa virheellisiä. Tämä osoittaa, että luonnollisen kielen tietokantarajapintojen arvioinnissa on tarkasteltava samanaikaisesti sekä oikeellisuutta että vaihtelua.

Kokonaisuutena tutkielma tukee alan yleistä näkemystä, jonka mukaan luonnollisen kielen käyttö tietokantojen rajapintana on lupaava mutta vielä osin epävarma lähestymistapa [21, 10]. Cortex Analyst voi toimia tehokkaana työkaluna erityisesti rajatuissa käyttötapauksissa, joissa tietokannan rakenne tunnetaan hyvin ja se

manttinen näkymä on laadittu huolellisesti. Sen sijaan tilanteissa, joissa kysymykset sisältävät implisiittisiä oletuksia, monitulkintaisuutta tai useita päättelyvaihteita, järjestelmän tuottamiin kyselyihin on syytä suhtautua varauksella.

Tutkielman tulokset viittaavat siihen, että käytännön kehitystyössä huomio kannattaa kohdistaa paitsi mallin suorituskykyyn myös siihen, miten järjestelmälle tarjottu semanttinen konteksti on rakennettu. Tulevassa tutkimuksessa olisi hyödyllistä vertailla useita erilaisia semanttisia näkymiä, testata järjestelmää muissa sovellusympäristöissä sekä tarkastella tuotettujen SQL-kyselyiden lisäksi myös niiden palauttamien vastausten oikeellisuutta. Lisäksi useamman arvioijan käyttö voisi parantaa luokittelun reliabiliteettia ja vahvistaa tulosten luotettavuutta.

Tämän tutkimuksen perusteella voidaan päätellä, että kielimallipohjaiset luonnollisen kielen tietokantarajapinnat eivät vielä täysin korvaa asiantuntijan laatimia kyselyitä, mutta ne voivat jo nyt toimia hyödyllisenä tukena analytiikkatyössä. Niiden käytännön arvo riippuu kuitenkin ratkaisevasti siitä, kuinka hyvin järjestelmän toimintaympäristö, semanttinen malli ja käyttötapa on sovitettu yhteen.

Viitteet

- [1] Anthropic. System card: Claude Sonnet 4. Technical report, Anthropic, March 2026.
- [2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [3] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. Neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.
- [4] Léon Bottou. Online learning and stochastic approximations. *Online Learning and Neural Networks*, 1998. Revised, October 2010.
- [5] Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018.
- [6] Shuaichen Chang, Jun Wang, Mingwen Dong, Lin Pan, Henghui Zhu, Alexander Hanbo Li, Wuwei Lan, Sheng Zhang, Jiarong Jiang, Xiang Deng, Kewei Tu, and Zhiguo Wang. Dr.Spider: A diagnostic evaluation benchmark towards robust text-to-SQL semantic parsing. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- [7] Anwoy Chatterjee, H S V N S Kowndinya Renduchintala, Sumit Bhatia, and Tanmoy Chakraborty. POSIX: A prompt sensitivity index for large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 14550–14565, 2024.
- [8] Lingjiao Chen, Matei Zaharia, and James Zou. How is ChatGPT’s behavior changing over time? *Harvard Data Science Review*, (Special Issue 5), 2024.
- [9] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does BERT look at? an analysis of BERT’s attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 277–286. Association for Computational Linguistics, 2019.
- [10] Naihao Deng, Yulong Chen, and Yue Zhang. Recent advances in text-to-SQL: A survey of what we have and what we expect. In *Proceedings of the 29th International Conference on Computational Linguistics (COLING)*, pages 2166–2187, 2022.
- [11] Xiang Deng, Ahmed Hassan Awadallah, Christopher Meek, Oleksandr Polozov, Huan Sun, and Matthew Richardson. Structure-grounded pretraining for text-to-SQL. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 1337–1350, 2021.

- [12] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.
- [13] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yuchen Zhu, Shi Chen, Zhi-Hong Deng, and Xing Zhang. Text-to-SQL empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363*, 2023.
- [14] Mor Geva, Roei Schuster, Jonathan Berant, and Dimitra Gkatzia. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5484–5495, 2021.
- [15] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [17] Dan Jurafsky and James H. Martin. *Speech and Language Processing*. Prentice Hall, 3rd ed. draft edition, 2023.
- [18] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2015.
- [19] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? a BIG bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS 2023)*, volume 36, 2024.
- [20] Seppo Linnainmaa. The representation of the cumulative rounding error of an algorithm as a taylor expansion of the local rounding errors. *Master’s Thesis, University of Helsinki*, 1970.
- [21] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. A survey of text-to-SQL in the era of LLMs: Where are we, and where are we going? *arXiv preprint arXiv:2408.05109*, 2024.
- [22] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2019.
- [23] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [24] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, Cambridge, MA, 1969.

- [25] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, New York, 2 edition, 2006.
- [26] Boris T Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.
- [27] Mohammadreza Pourreza and Davood Rafiei. ETM: Modern insights into perspective on text-to-SQL evaluation in the age of large language models. *Future Internet*, 17(8):325, 2025.
- [28] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018.
- [29] Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of adam and beyond. *arXiv preprint arXiv:1904.09287*, 2018.
- [30] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [31] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.
- [32] Frank Rosenblatt. *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Spartan Books, Washington, DC, 1962.
- [33] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.
- [34] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- [35] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3–4):379–423, 623–656, 1948.
- [36] Claude E. Shannon. Prediction and entropy of printed english. *Bell System Technical Journal*, 30(1):50–64, 1951.
- [37] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4596–4604. PMLR, 2018.
- [38] Snowflake Inc. Semantic model specification for Cortex Analyst, 2024. Viitattu: 24.8.2026.
- [39] Snowflake Inc. Cortex analyst. <https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-analyst>, 2026. Retrieved April 22, 2026.

- [40] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [41] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *International Conference on Machine Learning*, pages 1139–1147. PMLR, 2013.
- [42] Tijmen Tieleman and Geoffrey Hinton. Lecture 6a - neural networks for machine learning. *COURSERA: Neural Networks for Machine Learning*, 2012. Lecture notes / Coursera course material.
- [43] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.
- [44] Paul J. Werbos. Backpropagation through time: What it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.
- [45] Paul John Werbos. *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. PhD thesis, Harvard University, 1974.
- [46] Andrew G. Wilson and P. others. Marginal likelihood and the evidence lower bound (elbo). *arXiv preprint*, 2017.
- [47] Ruibin Xiong, Yichun Yang, Di He, Kaihe Zheng, Shuxin Zheng, Chenglin Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 10524–10533. PMLR, 2020.
- [48] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [49] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3911–3921, 2018.
- [50] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.